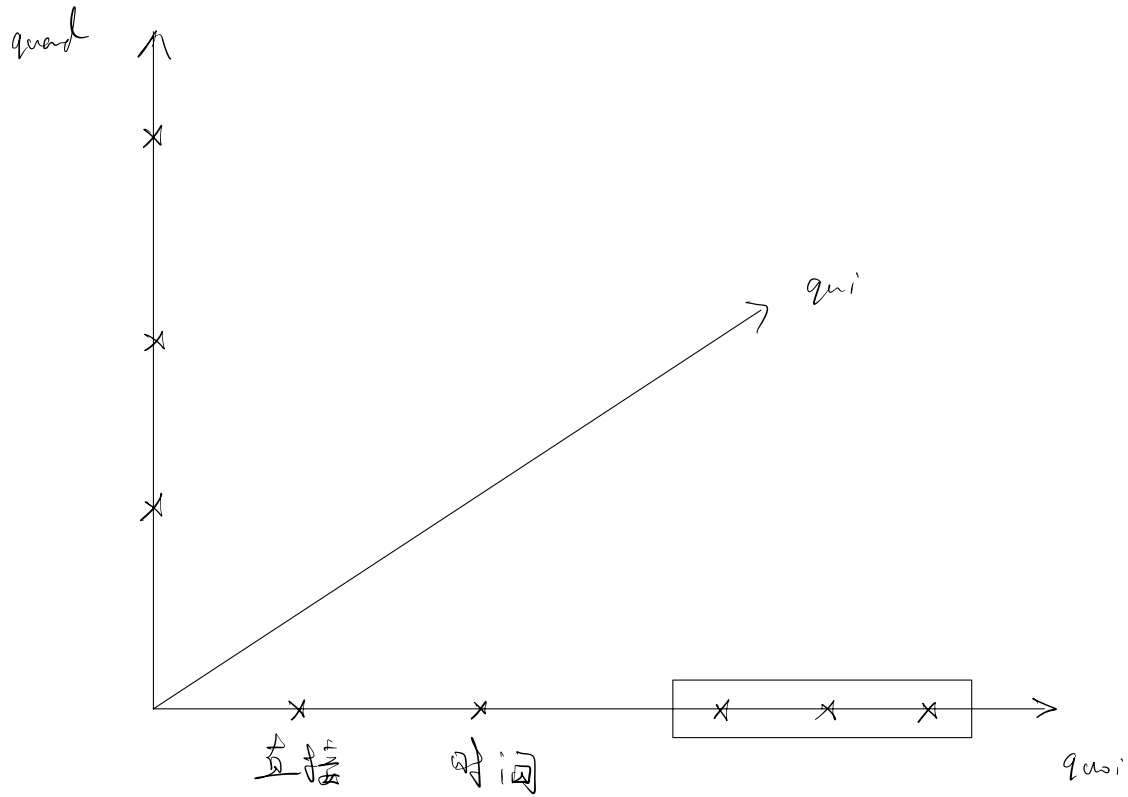
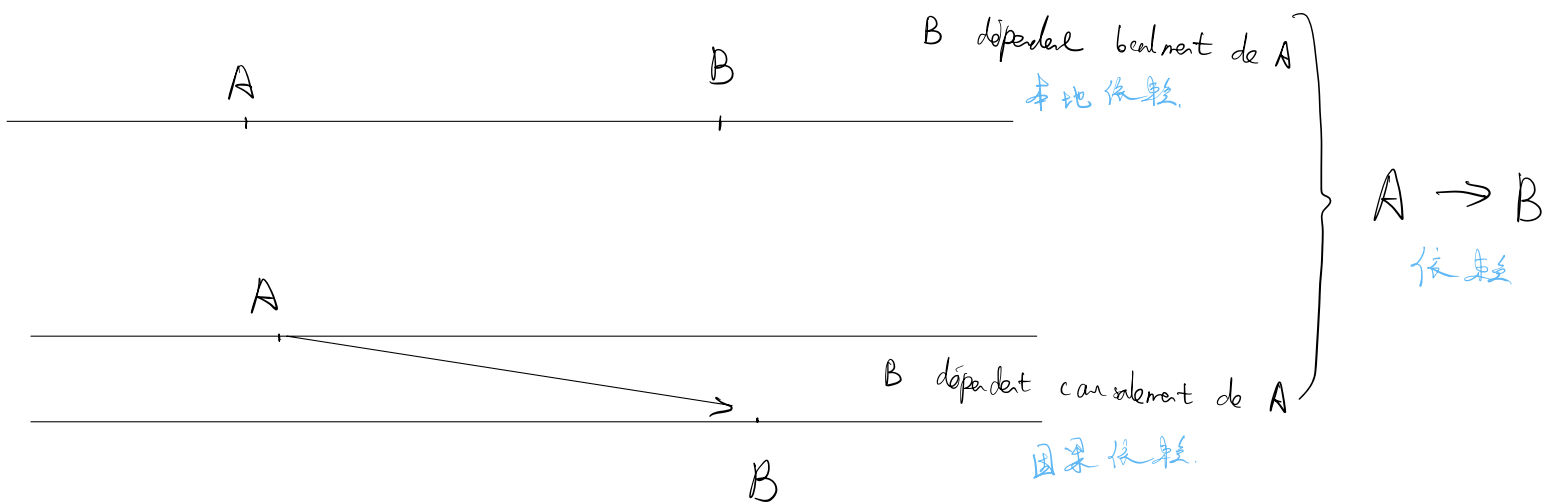


what, when, who 模型

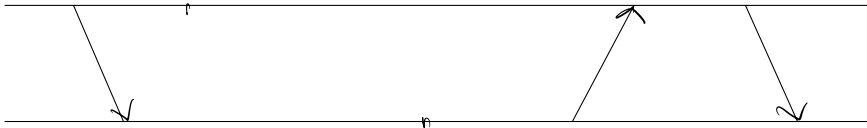


$\mathbb{R}^B$

解占座



A

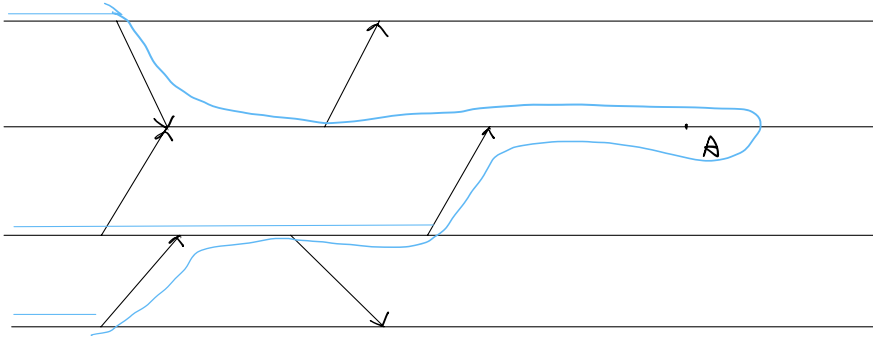


B

A et B indépendante

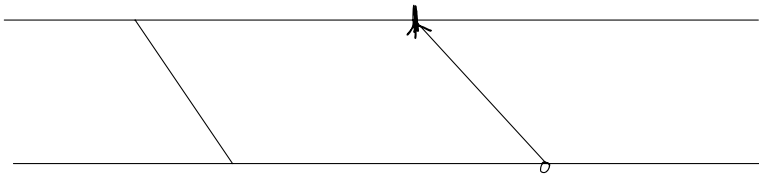
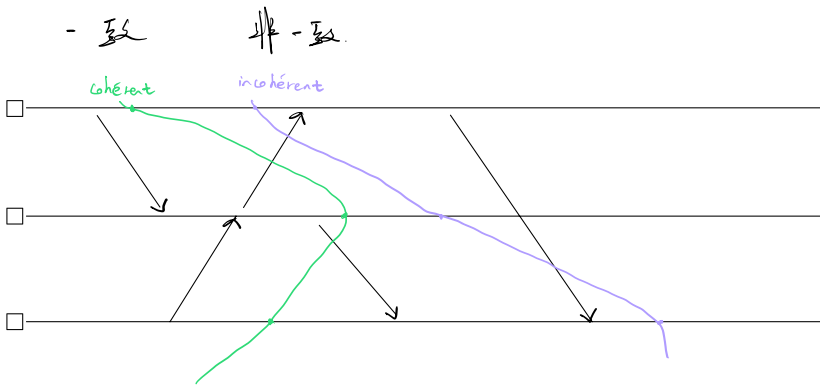
$A \parallel B$

平行



$\text{Hist}(A) \{x, x \Rightarrow A\}$

依軌跟踪

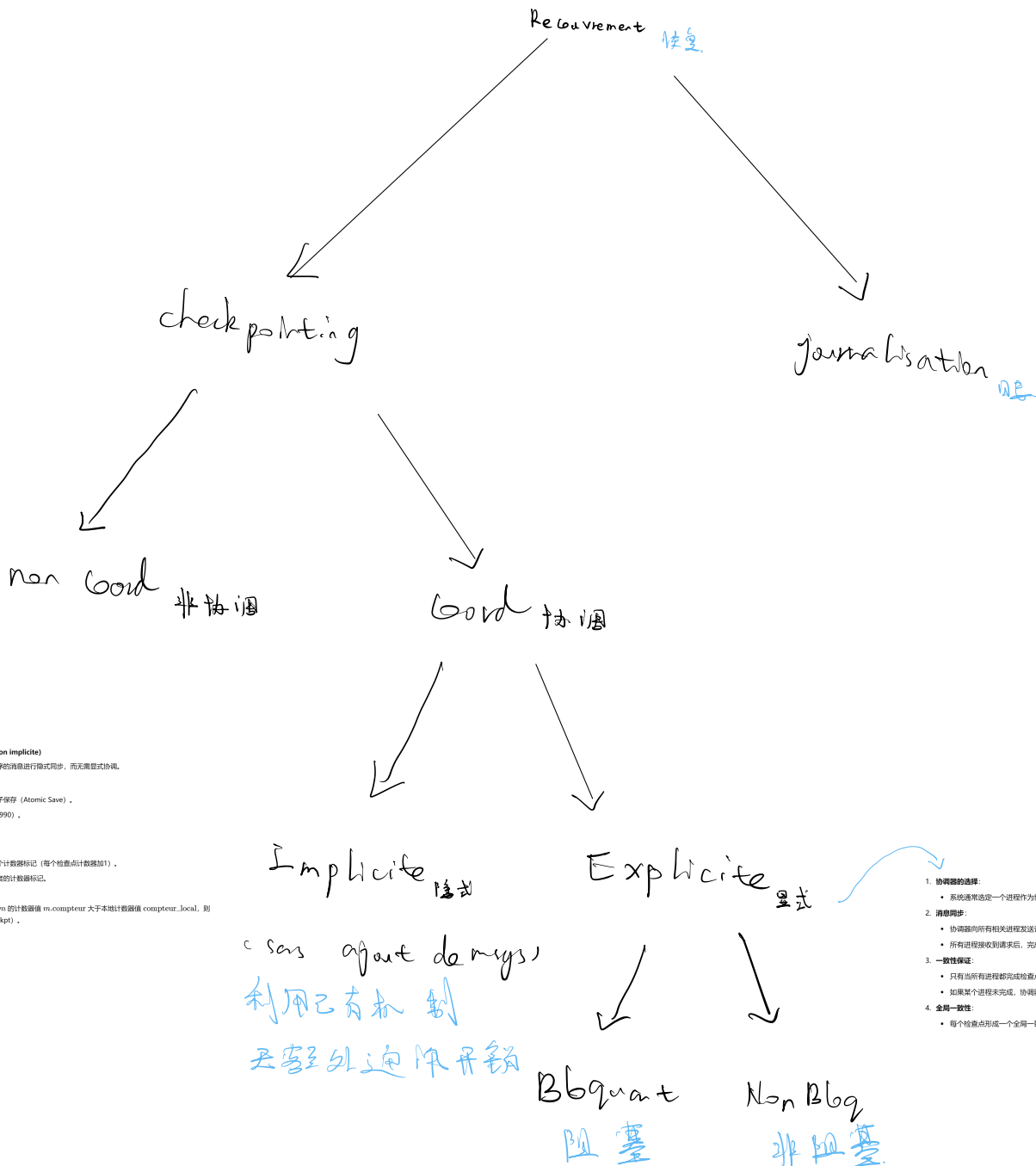


Carpeen coherence

- Tous les message emis dans le passé
sont reçu dans le passé, sent le ressource.
- Tous les message emis dans le future
doivent être reçu dans le future.

1. 一致性条件 (Conjure coherent) :

- 条件 1:
 - 所有在**过去**发送的消息, 必须在**过去**被接收。✓
 - 解释: 系统中不存在“孤立消息”或未被处理的历史消息。
- 条件 2:
 - 所有在**未来**发送的消息, 必须在**未来**被接收。✓
 - 解释: 系统需要保证消息传递的因果性。



隐式同步协调 (Coordination implicite)

- 核心思想: 利用应用程序的消息进行隐式同步, 而无需显式协调。

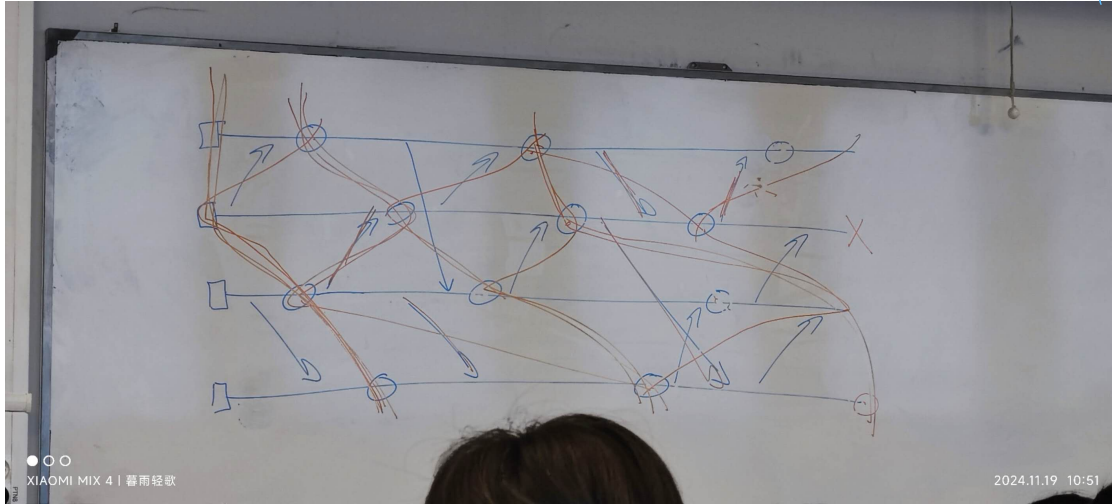
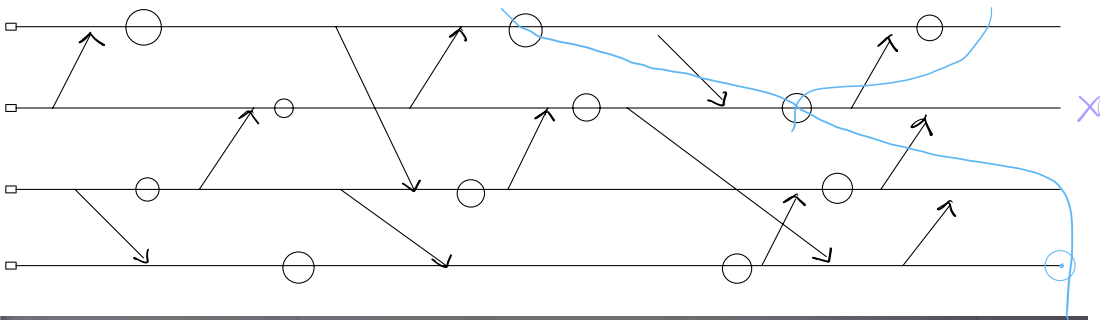
算法 "naïf" (朴素算法)

- 每次消息发送后进行原子保存 (Atomic Save)。
- 来源: Wu & Fuchs (1990)。

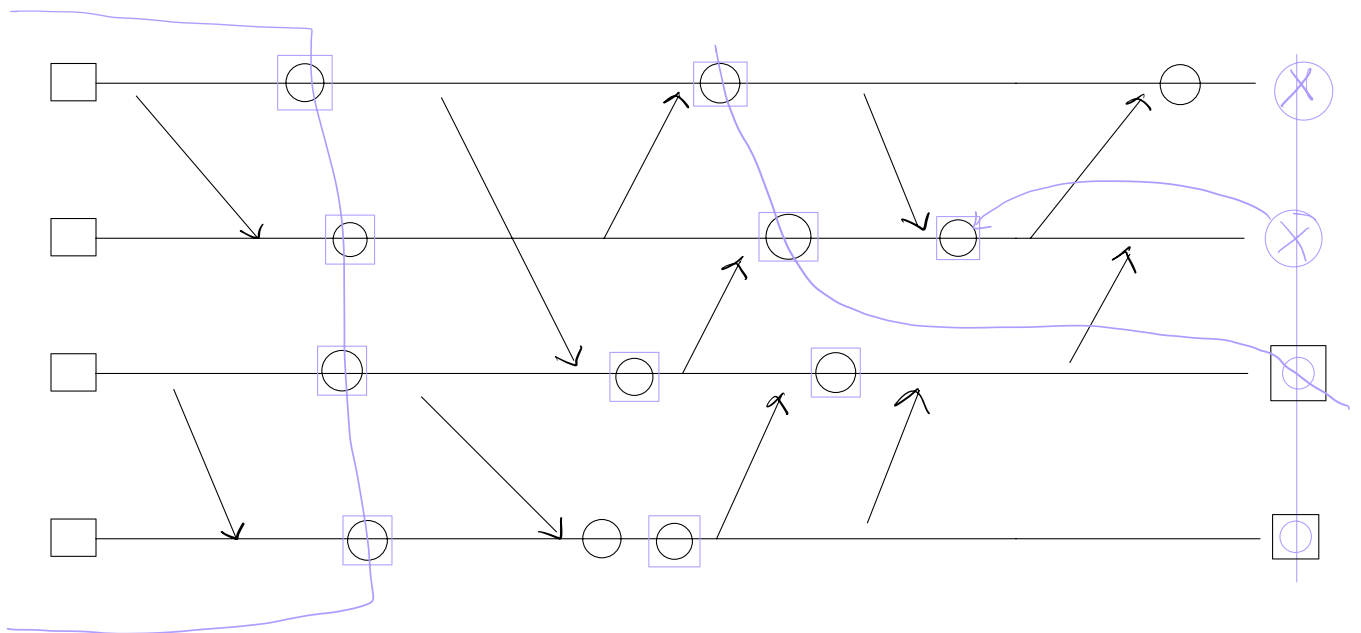
基础算法 (Briatico B4)

1. 计数器机制:
 - 每个检查点都由一个计数器标记 (每个检查点计数器加1)。
 - 每条消息都被发送者的计数器标记。
2. 消息接收规则:
 - 如果接收到的消息 m 的计数器值 $m.compteur$ 大于本地计数器值 $compteur_local$, 则触发本地的检查点 (ckpt)。

1. 协调器的选择:
 - 系统通常选是一个进程作为协调器, 负责发起和管理检查点操作。
2. 消息同步:
 - 协调器向所有相关进程发送请求消息, 要求它们保存当前状态并暂停某些活动。
 - 所有进程接收到请求后, 完成保存并确认。
3. 一致性保证:
 - 只有当所有进程都完成检查点的保存后, 系统才会继续运行。
 - 如果某个进程未完成, 协调器可能重试或中止协调。
4. 全局一致性:
 - 每个检查点形成一个全局一致的状态快照, 确保恢复时系统能够回溯到一致的状态。



algo check point graphe



$\forall \alpha \text{ coupure} \in T, \text{ tous états admetts à blanc}$

$\forall \alpha \text{ gris}, T \text{ à les sites crash}$

$Tq \exists 1 \text{ gris de coupure}$

$\times$ pour chaque état qui

$\rightarrow$ remplace perte part en blanc

$\times$ pour chaque nouvel état E

$\rightarrow$ gris les site x et $q \in E \rightarrow x$

1. 第一部分:

- " $\forall \alpha \text{ coupure} \in T, \text{ tous états admet à blanc}$ "
- 翻译: 对于所有属于 T 的 α 切断点, 所有状态需要接受为"白板"或初始一致状态。

2. 第二部分:

- " $\forall \alpha \text{ gris}, T \alpha \text{ les sites crash}$ "
- 翻译: 对于所有灰色状态, $T\alpha$ 表示崩溃的站点集合。

3. 第三部分:

- " $Tq \exists 1 \text{ gris de coupure}$ "
- 翻译: 对于每个 Tq , 存在一个灰色的切断状态。

4. 第四部分:

- " $\times$ pour chaque état grisé $\rightarrow$ remplace perte part en Blanc"
- 翻译: 对于每个灰色状态, 需要替换丢失部分为白板状态。

5. 第五部分:

- " $\times$ pour chaque nouvel état $E \rightarrow$ grise les états $X \forall y \in E \rightarrow X$ "
- 翻译: 对于每个新状态 E , 使状态 X 变为灰色, 且对于所有 y 满足 E 导致 X 。

1. 白色 (Blanc) : 一致状态

- 定义: 白色状态表示一个干净、确定且一致的状态, 是系统恢复和继续运行的安全起点。
- 特点:
 - 没有未处理的中间状态。
 - 不会受到其他外部依赖的影响。
 - 可作为系统一致性恢复的基础。

• 实现方式:

- 在系统中, 白色通常由 **检查点 (Checkpoint)** 实现。这些检查点记录了分布式系统中每个节点在某一时间点的一致状态。
- 在需要恢复时, 系统会回滚到最近的白色状态。

2. 灰色 (Gris) : 不一致或中间状态

- 定义: 灰色状态是一种中间状态, 表示系统尚未完全处理完某些事件, 或者某些操作仍在传播中 (可能引起不一致)。
- 特点:
 - 部分完成: 操作或事件还未完全结束。
 - 依赖问题: 状态可能依赖其他节点或事件, 尚未同步。
 - 潜在危险: 可能导致系统处于不一致或失败 (crash) 状态。
- 来源:
 - 灰色状态通常发生在系统接收到新的消息但还未完成处理时。

1. 第一部分:

$\forall \alpha \text{ coupure} \in T, \text{ tous états admet à blanc}$

解释:

- 对属于集合 T 的每个"切断点", 所有相关状态都必须被初始化为"白色", 即一致的基准状态。
- 这一部分表明分布式系统中的一致性需要确保所有的初始条件得以恢复。

2. 第二部分:

$\forall \alpha \text{ gris}, T \alpha \text{ les sites crash}$

解释:

- 对于所有处于"灰色"的状态 (即部分处理但可能不一致的中间状态), 对应的节点 (sites) 被认为是潜在的失败点 (crash)。

3. 第三部分:

$Tq \exists 1 \text{ gris de coupure}$

解释:

- 对于每个 Tq , 至少存在一个灰色的切割点。
- 这是指系统在某些切割点上可能存在不一致的中间状态, 且需要特别处理。

4. 第四部分:

$\times$ pour chaque état grisé $\rightarrow$ remplace perte part en Blanc

解释:

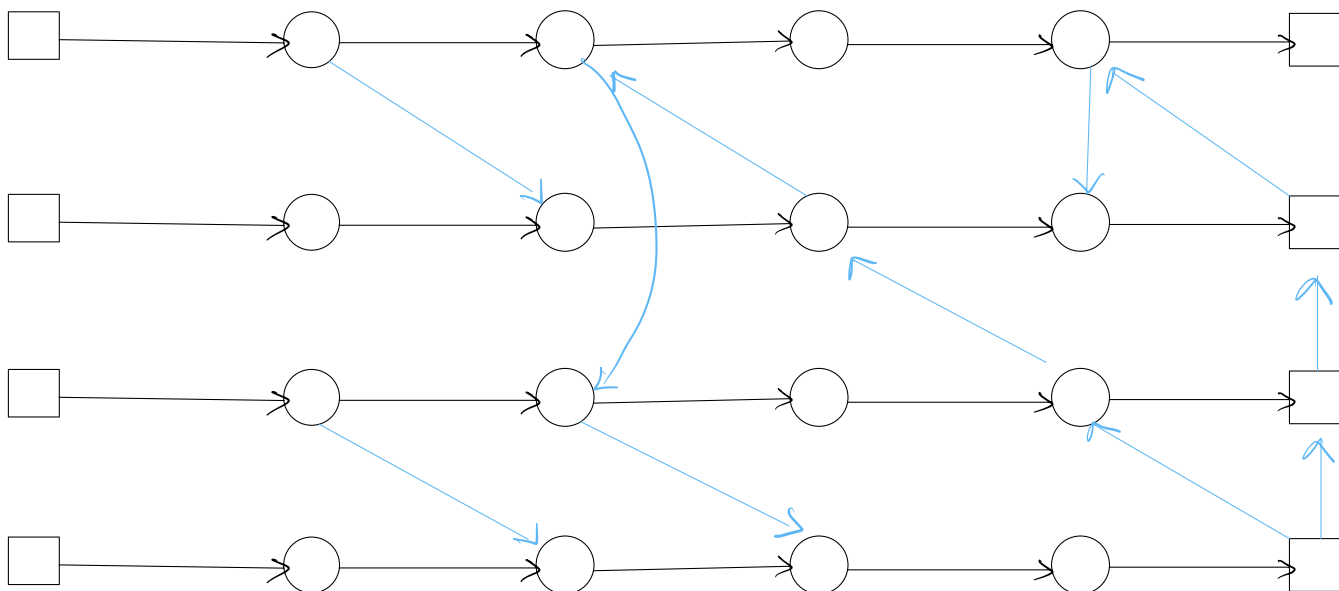
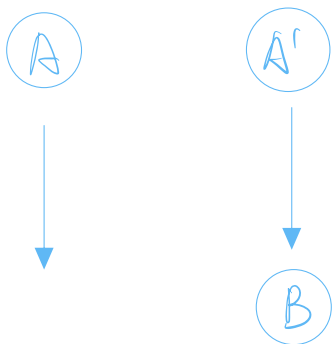
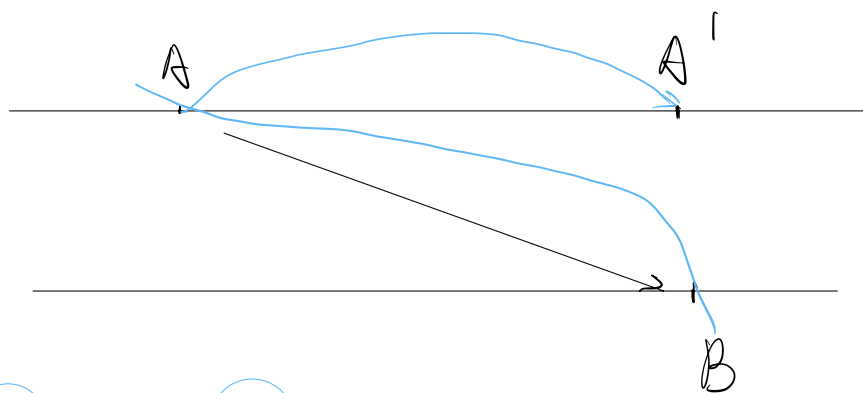
- 对于每一个灰色状态, 需要将其转换为"白色"或初始状态。
- 表明任何可能影响系统一致性的中间状态必须被回滚以恢复到一个确定性状态。

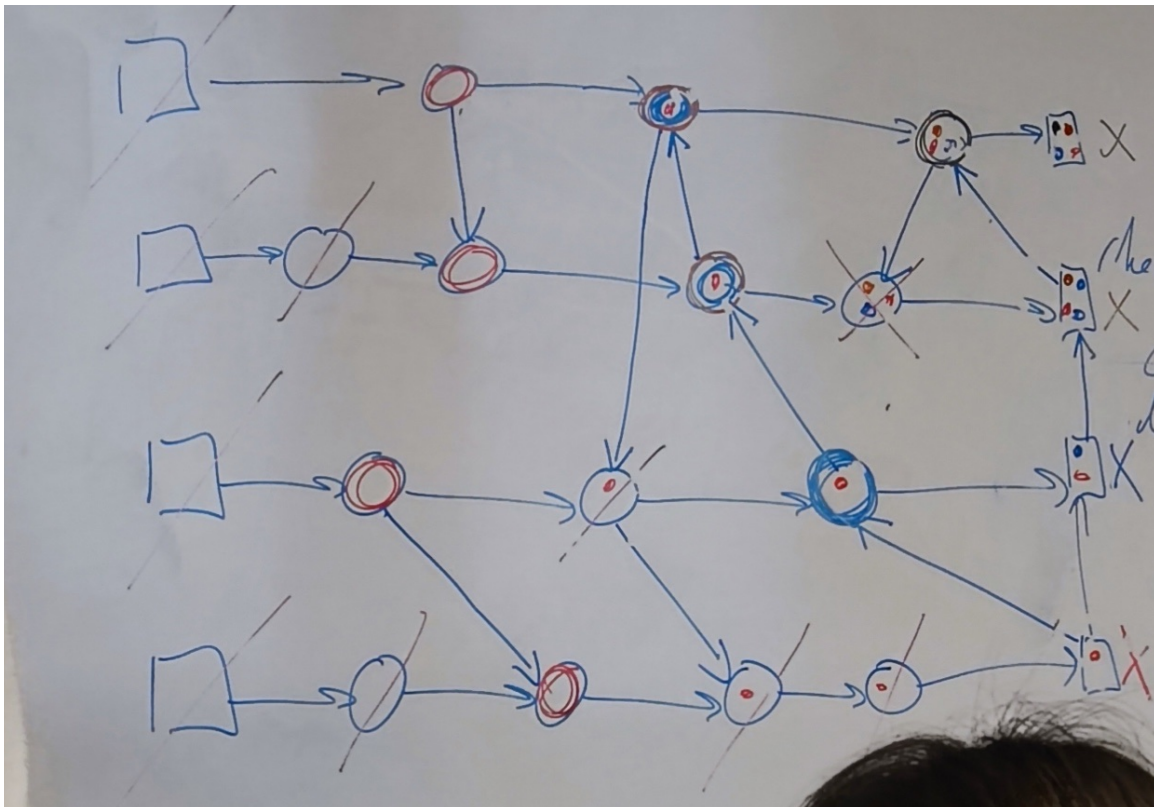
5. 第五部分:

$\times$ pour chaque nouvel état $E \rightarrow$ grise les états $X \forall y \in E \rightarrow X$

解释:

- 对于每一个新生成的状态 E , 如果它由灰色状态 X 影响, 那么需要将 E 标记为灰色
- 这描述了一种"灰色传播"的机制, 即中间不一致状态的影响会扩散到新状态。





$\exists$ path entre A et B

$$A \xrightarrow{Z} B$$

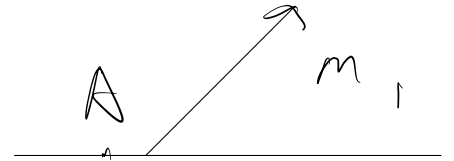
une séquence de msg

$$\{m_1, \dots, m_n\}$$

est z chemin de A à B

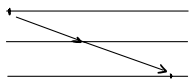
ssi

① m_1 émis un site de après A

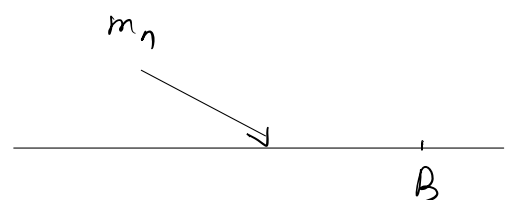


② $\forall i$ reçu m_i , un $\hat{m}$
n'est que send de m_{i+1}

Et pas de checkpoint entre
reçu m_i , et send m_{i+1} ,



③ m_n reçu le site de B, avant B



图片 1：Z路径定义

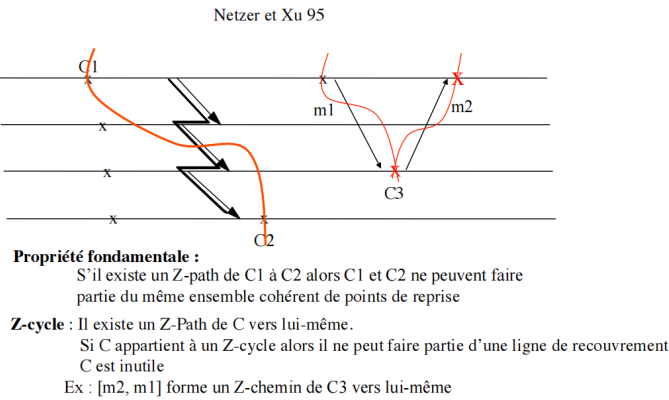
1. 核心概念：
- “ $\exists$ path entre A et B”：表示存在从节点 A 到节点 B 的路径。
 - “Z”路径：是由一组消息 $\{m_1, m_2, \dots, m_r\}$ 构成的路径，从 A 到 B。
 - 关键在于描述消息传播的过程以及这些消息如何在检查点和非一致状态之间形成路径。
2. 解释：
- Z路径是分布式系统中因果关系的体现。
 - 如果一个节点 A 的状态（通过某些消息 $m_1, m_2, \dots, m_r$ ）对 B 造成影响，这些消息链路形成了一个 Z路径。
3. 课件内容关联：
- 课件【12】提到 Zigzag Path 的定义：消息 m_1 从 A 发出，传播到 B 前经过多个中间节点，如果这些节点间没有一致性检查点，就可能形成潜在的 Z路径。

图片 2：Z路径的条件

1. 条件 1：消息的发送：
- “m1 émis un site de après A”：
 - m_1 是从 A 的某个状态发出的消息。
 - 状态 A 是 Z路径的起点。
2. 条件 2：中间消息传递：
- “Vi reçu mi, un mi+1 n'est que send de mi+1”：
 - 当节点接收到消息 m_i 后，它只会进一步发送消息 m_{i+1} 。
 - 在消息接收和发送之间，不能有任何检查点。
 - 解释：
 - 这表明消息的中间传播必须是连续的，没有额外的状态保存操作。
3. 条件 3：最终到达：
- “mn reçu le site de B, avant B”：
 - 最后一条消息 m_r 被节点 B 接收，形成路径的终点。
4. 总结：
- 这三个条件共同定义了 Z路径的完整性，即消息必须在没有检查点中断的情况下，从 A 持续传播到 B。

Zigzag-Path (Xu & Netzer 93)

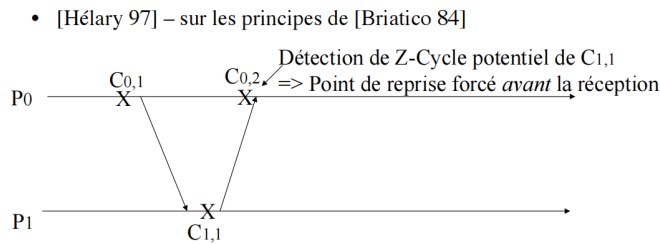
- Extension des chemins de causalité de Lamport
- Un Zigzag chemin (Z-path) de $C_{p,i}$ à $C_{q,j}$ est une séquence de messages $(m_1, m_2, \dots, m_l)$; $l \geq 1$, telle que:
 - m_1 est envoyé par p après $C_{p,i}$
 - Si m_k ($1 \leq k < l$) est reçu par r , alors m_{k+1} est envoyé par r dans le même intervalle (ou plus tard)
 - m_{k+1} peut être envoyé après la réception de m_k
 - m_l est reçu par q avant $C_{q,j}$

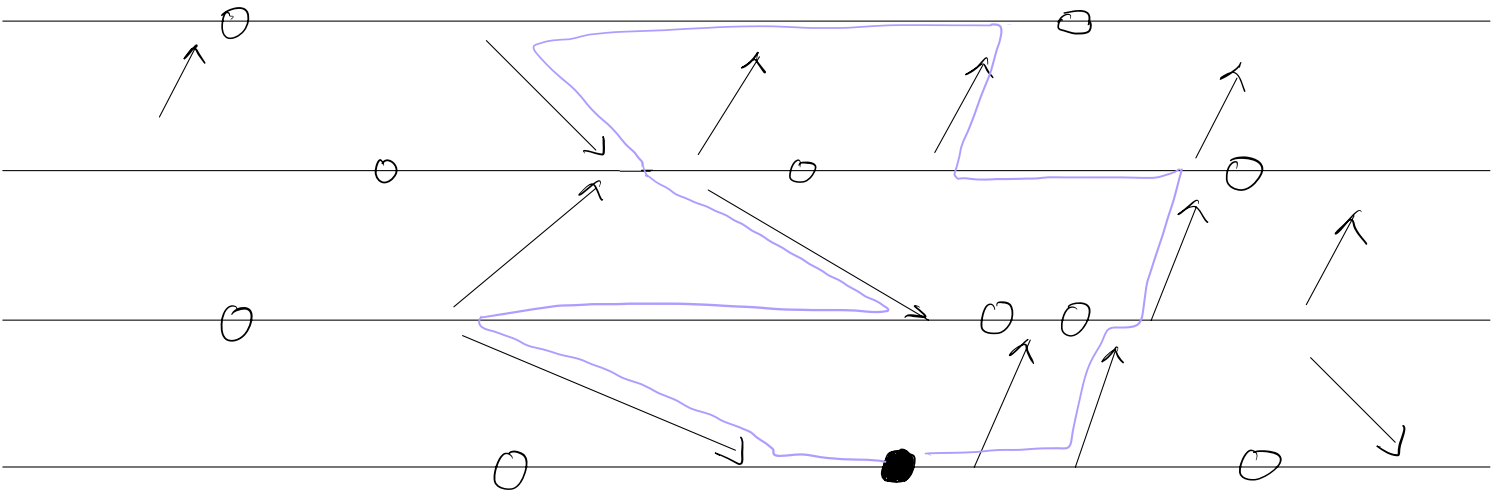


Zigzag 路径 (Z-path) 的定义：

从 $C_{p,i}$ 到 $C_{q,j}$ 的 Zigzag 路径是一系列消息 $(m_1, m_2, \dots, m_l)$ ，其中 $l \geq 1$ ，满足以下条件：

1. 起点条件：
- 消息 m_1 是在检查点 $C_{p,i}$ 之后由进程 p 发送的。
2. 中间转发条件：
- 如果消息 m_k ($1 \leq k < l$) 被进程 r 接收，则消息 m_{k+1} 是由进程 r 在相同时间间隔（或更晚）发送的。
 - m_{k+1} 可以在接收 m_k 后立即发送，也可以稍晚发送。
3. 终点条件：
- 最后一条消息 m_l 在检查点 $C_{q,j}$ 之前由进程 q 接收。





图片 3：具体的 Z 路径示意图

1. 图中元素：
- 水平线：表示时间线或状态演化。
 - 圆圈：表示不同的状态点（包括检查点）。
 - 蓝色路径：表示 Z 路径的传播。
2. 分析：
- Z 路径从某个状态点（左侧）出发，通过消息传播跨越多个节点。
 - 在传播路径上，中间可能存在多条消息传递，但没有一致性检查点。
 - 右下角黑点表示路径的终点（类似条件 3 中的 B）。
3. 风险与影响：
- 如果 Z 路径中的某一状态发生故障（如黑点对应的状态），会引发回滚，并可能导致“效应扩散”（Effet domino）。
 - 这就是为什么需要检测并避免 Z 路径的原因。

1. Initialization (Init sur S_i):

$$\forall j, V_{S_i}[j] = \emptyset$$

表示在节点 S_i 上，对所有的 j ，初始化 $V_{S_i}[j]$ 的值为空集合 ($\emptyset$)。

2. Send (Send sur S_i):

$$V_{S_i}[i]++$$

表示在节点 S_i 上，对自己的版本号（或标识） $V_{S_i}[i]$ 进行自增操作。

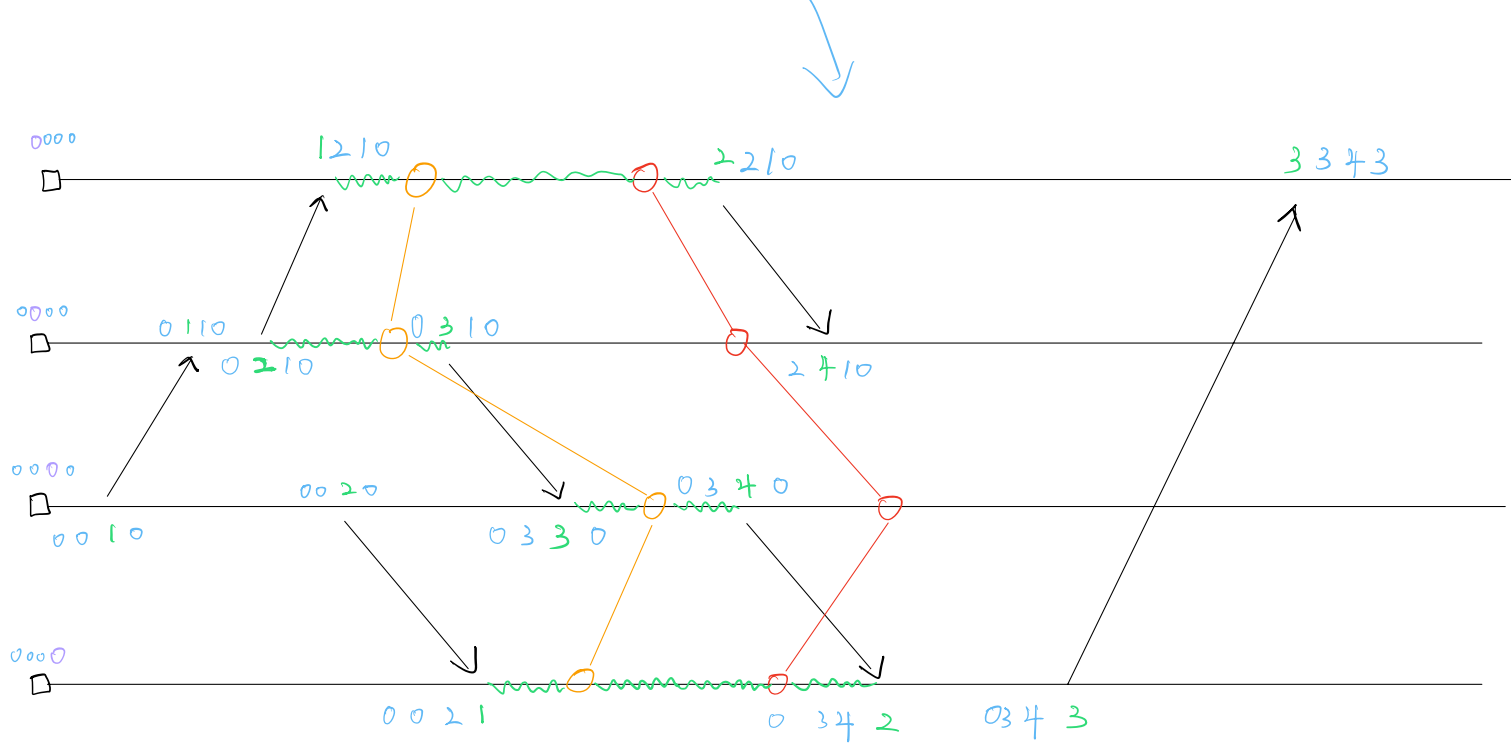
3. Receive (Reco sur S_i):

$$\begin{cases} V_{S_i}[j]++ \\ \forall k, V_{S_i}[k] = \max(V_{S_i}[k], V[j][k]) \end{cases}$$

这个部分说明节点 S_i 接收到来自某个 j 的信息后需要：

- 自增本地存储的 $V_{S_i}[j]$ 。
- 更新所有 k 对应的 $V_{S_i}[k]$ 值，取本地和远程版本号 $V[j][k]$ 的最大值。

向量时钟
规则



1 2 1 0
 0 2 1 0
 0 3 3 0
 0 0 2 1



1 3 3 1

1 2 3 1
=

1 2 1 0
 0 3 1 0
 0 3 4 0
 0 0 2 1

1 3 4 1

$$\forall i \quad V_{S_i}[i] = \text{reervoir}_{S_i}(V_{S_j}[i])$$

$$\parallel \quad C_a \rightarrow C_b \quad V_{C_a} < V_{C_b}$$

$$\parallel \quad C_a \parallel C_b \quad \vdash (V_{C_a} < V_{C_b}) \wedge \vdash (V_{C_b} < V_{C_a})$$

1. 第一行：更新向量时钟规则

$$\forall i, V_{S_i}[i] = \text{reervoir}_{S_i}(\cup V_{S_j}[i])$$

- **解释：**
 - 对于节点 S_i ，其向量时钟中的第 i 个分量通过接收消息中的时间戳更新。
 - reervoir_{S_i} 表示节点 S_i 从其他节点 S_j 接收到的消息。
 - $\cup V_{S_j}[i]$ 是节点 S_j 的向量时钟的第 i 个分量的联合集。
- **作用：**
 - 这是向量时钟更新规则的数学表达。
 - 当节点 S_i 接收到从其他节点发送的消息时，它会将自身的向量时钟与消息中的向量时钟逐一比较，并取较大值更新自己的时钟。
- **背景：**
 - 在分布式系统中，这一规则确保接收消息的节点可以更新对系统其他节点的了解，保持因果一致性。

2. 第二行：因果关系的定义

$$C_a \rightarrow C_b \iff V_{C_a} < V_{C_b}$$

- **解释：**
 - $C_a \rightarrow C_b$ ：表示事件 C_a 因果地发生在 C_b 之前。
 - $V_{C_a} < V_{C_b}$ ：向量时钟 V_{C_a} 的所有分量都小于或等于 V_{C_b} ，并且至少有一个分量严格小于。
- **作用：**
 - 这是向量时钟用于判定因果关系的核心规则。
 - 如果 C_a 的时钟小于 C_b ，说明 C_b 的发生受到了 C_a 的影响（即 C_a 是 C_b 的因）。
- **背景：**
 - 这是 Lamport 因果关系模型的扩展。
 - 向量时钟提供了比标量时钟更精确的因果关系判定能力。

3. 第三行：并发关系的定义

$$C_a \parallel C_b \iff \neg(V_{C_a} < V_{C_b}) \wedge \neg(V_{C_b} < V_{C_a})$$

- **解释：**
 - $C_a \parallel C_b$ ：表示事件 C_a 和 C_b 是并发的。
 - **条件：**
 - $\neg(V_{C_a} < V_{C_b})$ ： C_a 的时钟不严格小于 C_b 。
 - $\neg(V_{C_b} < V_{C_a})$ ： C_b 的时钟也不严格小于 C_a 。
- **作用：**
 - 如果 C_a 和 C_b 既不是因果关系（ $C_a \rightarrow C_b$ 或 $C_b \rightarrow C_a$ ），则它们是并发的。
 - 并发关系表明两个事件相互独立，可能同时发生。
- **背景：**
 - 并发关系是分布式系统中重要的概念，它强调事件之间没有直接的因果依赖。

1. $V_A \leq V_B$:

$$V_A \leq V_B \iff \forall i, V_A[i] \leq V_B[i]$$

这是向量时钟中两个向量之间的“部分顺序”关系定义。表示如果 V_A 的每个分量都小于或等于 V_B 的对应分量，则认为 $V_A \leq V_B$ 。

2. $V_A = V_B$:

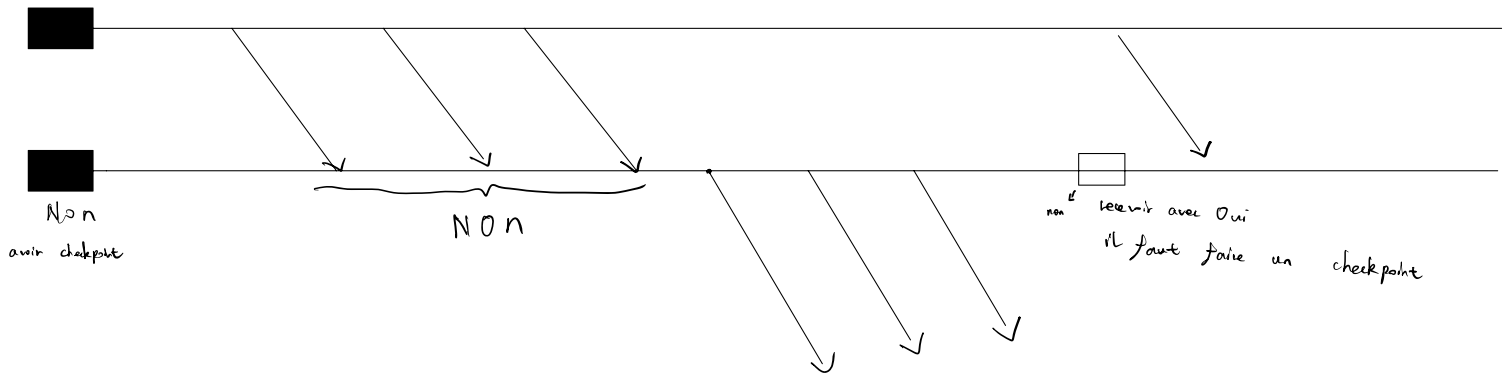
$$V_A = V_B \iff \forall i, V_A[i] = V_B[i]$$

如果两个向量在所有分量上都相等，则它们是相等的。

3. 严格小于 (Causal Happens-before关系) $V_A < V_B$:

$$V_A < V_B : V_A \leq V_B \text{ 且 } \exists j, V_A[j] < V_B[j]$$

表示 V_A 严格小于 V_B ，不仅每个分量都小于或等于对应分量，还要求至少存在一个分量 j ，使得 $V_A[j] < V_B[j]$ 。这与分布式系统中事件的因果关系 ($A \rightarrow B$) 密切相关。



init $vec \leftarrow faire$

send

le receiver

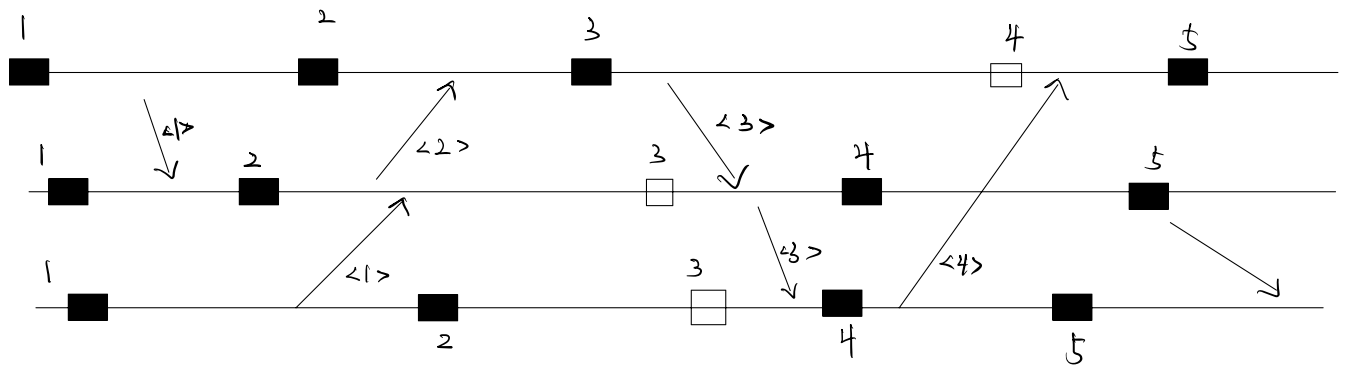
if

condition

checkpoint

Rond 80

Briatico 84



recevoir if $H_{bc} < H_{msg}$

check point

$H_{bc} \leftarrow H_{msg}$

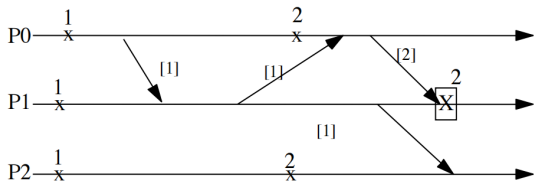
Algorithme de base : [Briatico 84]

Un compteur date les points de reprise (+1 à chaque checkpoint)

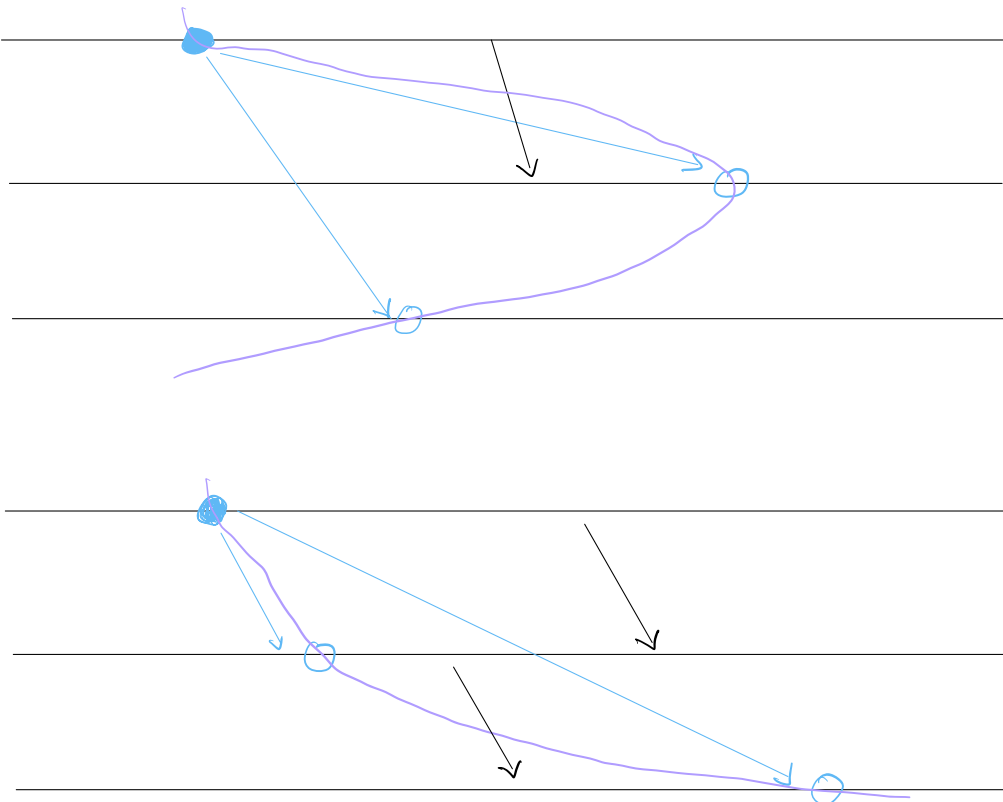
Chaque message est estampillé par le compteur de l'émetteur

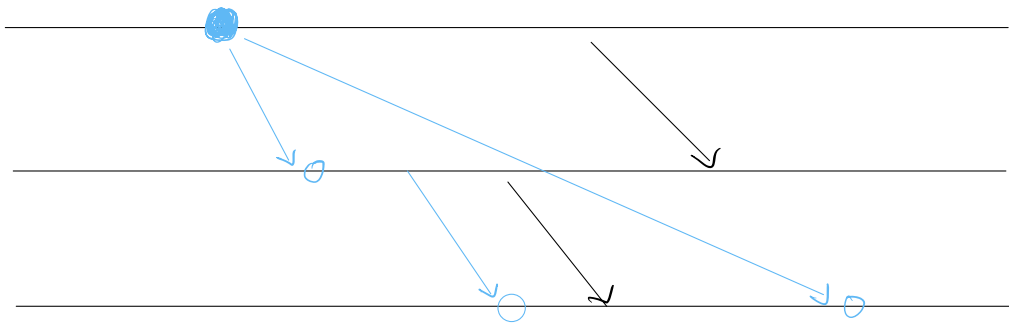
Réception d'un message m :

ckpt si $m.compteur > compteur_local$



Fifo





1. Chandy-Lamport 85

- 假设：信道是 FIFO (First-In-First-Out)
- 机制：
 1. **检查点请求 (Ckpt_req)** 由一个进程 (Initiateur) 发送给所有进程。
 2. 每个进程在收到 Ckpt_req 后进行检查点存储，然后广播一个标记 (Marker)。
 3. **标记 (Marker)** 用于识别所有在检查点之间的消息，保证一致性。
 4. 所有在收到 Marker 之前接收的消息属于当前检查点前的状态，而之后的消息属于下一个检查点后的状态。
 5. 由于 FIFO 假设，接收到的 Marker 之前的消息是完整的，这样可以确保快照的一致性。

✅ 优点：

- 在 FIFO 信道的假设下，确保所有进程记录的一致性检查点是正确的，避免了不一致的状态。

❌ 缺点：

- 需要信道是 FIFO，否则可能会导致消息乱序，影响一致性。
- 额外的消息开销：每个进程在存储检查点时，都要发送 Marker。

2. Lai-Yang 87

- 假设：信道是 非 FIFO (Non-FIFO)

• 机制：

1. 检查点请求 (Ckpt_req) 仍然由 Initiateur 发送。
2. 进程不直接记录检查点，而是根据消息传递动态调整。
3. 消息 `m` 可以乱序到达，因此在接收消息 `m` 时，可能需要回溯调整检查点，以确保检查点存储的一致状态。
4. 允许非 FIFO 的通信信道，因此需要额外的处理来追踪消息的传递，保证所有进程的一致状态。

✅ 优点：

- 不需要信道是 FIFO，适用于更加通用的分布式系统。
- 更少的 Marker 消息，减少了 Chandy-Lamport 算法的通信开销。

❌ 缺点：

- 由于消息可能乱序传输，需要额外的逻辑来检测和调整检查点，可能导致更多的计算复杂度。

2. 需要增加的机制（从 Chandy-Lamport 85 到 Lai-Yang 87）

为了适应 非 FIFO 信道，可以引入以下机制：

(1) 记录消息的“逻辑时间戳”

在 Lai-Yang 87 中，每个消息 `m` 需要附带一个逻辑时间戳，用来标识它发送的顺序。这可以通过 向量时钟 (Vector Clocks) 或 Lamport 时钟 (Lamport Timestamps) 来实现：

- 每个进程在发送消息时，都会附带当前的逻辑时间戳。
- 每个进程接收消息后，会更新自己的逻辑时间，并确保检查点的一致性。

✅ 改进点：

- 允许消息乱序到达，因为每个消息的时间戳可以帮助我们正确分类它属于检查点前还是检查点后。

(2) 消息“标记”机制

在 Chandy-Lamport 85 中，进程在收到 Ckpt_req 之前就已经收到的消息（即检查点前的消息）会自动保留。但在 非 FIFO 信道下，一些旧消息可能会在 Ckpt_req 之后才到达，因此需要显式地记录：

1. 每个进程在记录检查点前，需要先标记所有当前处于等待状态的消息。
2. 所有未标记的消息（之后到达的）需要额外存储，并在恢复时重新处理。

✅ 改进点：

- 避免“幽灵消息” (Orphan Messages)。
- 允许消息乱序到达，确保即使消息先到达，检查点也能正确记录。

Lai - Yang 87

A : Atomicity 原子

C : Consistency 一致

I : Isolation 隔离

D : Durability 持久

Thomas CAP

不可能

Consistency

一致性

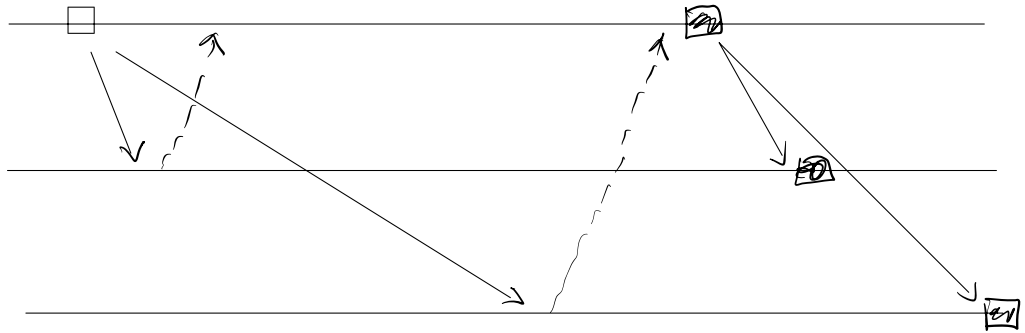
同时满足

A availability

可用性

P partition Tolerance 分区容忍性

sync and s logn



① checkpointing Registration / Ready 注册

② checkpoint Do / Done 执行

③ checkpoint Commit 提交

Point de reprise coordonnés : Sync-and-Stop (SaS)

Un coordinateur appelle une barrière de synchronisation pour geler l'application (arrêt des envois de message)

Algorithme :

1. Le coordinateur diffuse un message **chkptReq** à tous les processus
2. Réception **chkptReq** sur p :
p stoppe l'application (plus d'envoi) et renvoie **chkptReady** au coordinateur
3. Lorsque le coordinateur a reçu **chkptReady** de tous les processus, il diffuse un message **chkptDo** et sauvegarde son état
4. Réception de **chkptDo** sur p :
p sauvegarde son état et renvoie **chkptDone**
5. Lorsque le coordinateur a reçu tous les **chkptDone**, il diffuse **chkptCommit** pour débloquer les processus

算法步骤

1. 协调者广播检查点请求 (chkptReq)

- 协调者向所有节点广播一个消息 **chkptReq**，通知它们准备创建检查点。
- 所有节点收到此消息后，停止处理应用程序的任务（冻结消息的发送和接收）。

2. 节点发送准备状态 (chkptReady)

- 每个节点在收到 **chkptReq** 后：
 1. 暂停应用程序的执行（停止消息传递）。
 2. 返回一个消息 **chkptReady** 给协调者，表示它已准备好创建检查点。

3. 协调者确认所有节点准备就绪

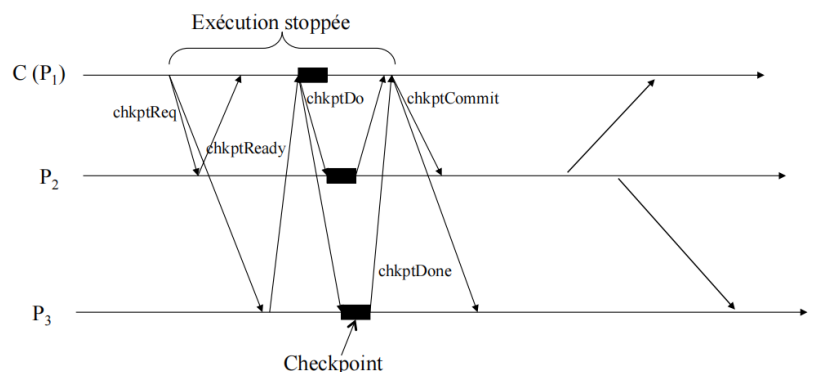
- 当协调者收到来自所有节点的 **chkptReady** 消息后，表示整个系统已准备好进行检查点操作。
- 它广播一个消息 **chkptDo**，指示所有节点保存检查点。

4. 节点创建检查点并确认完成

- 每个节点在接收到 **chkptDo** 后：
 1. 保存当前状态到检查点。
 2. 返回一个消息 **chkptDone** 给协调者，表示检查点操作已完成。

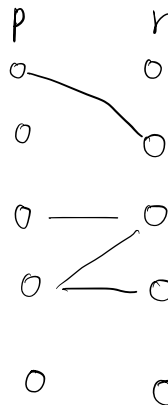
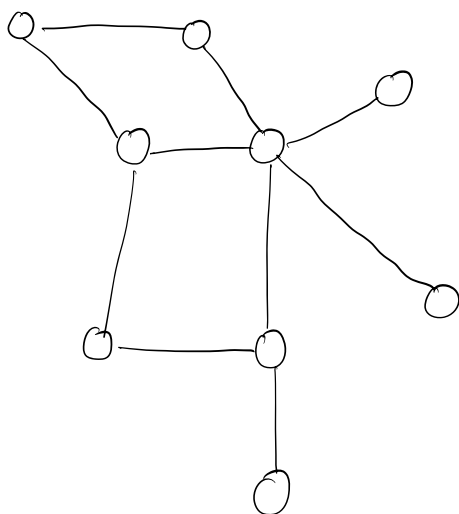
5. 协调者释放所有节点

- 当协调者收到所有节点的 **chkptDone** 消息后：
 1. 它广播一个 **chkptCommit** 消息。
 2. 所有节点恢复应用程序的正常操作。



M I S

Max Independent set



1 模型与定义

模型

我们将网络表示为一个无向连接图 $G = (V, E)$, 其中 V 是处理器的集合, E 是二元关系, 表示两个处理器能够通信的能力。如果 $(p, q) \in E$, 则 p 和 q 是邻居。每个处理器 p 能够识别其邻居并标记其局部标签, 且我们假设 p 维护了 N_p , 即其邻居本地标签的集合。以下, n 表示处理器的数量, D 表示最大度。如果 p 和 q 是网络中的两个处理器, 我们用 $d(p, q)$ 表示它们之间的最短路径长度 (即从 p 到 q 的距离)。

我们考虑经典的局部共享内存模型, 在该模型中, 邻居之间的通信通过变量的直接读取而非消息交换实现。在该模型中, 每个处理器由一组共享变量 (以下简称为变量) 建模, 并配有一组规则的集合。每个处理器只能写入自己的变量, 同时可以读取自己和邻居的变量。每条规则由以下形式组成:

$\langle \text{label} \rangle :: \langle \text{guard} \rangle \rightarrow \langle \text{statement} \rangle$

规则的标签主要用于区分它们。程序中的规则守卫是一个布尔谓词, 涉及处理器及其邻居的变量。规则的语句会更新一个或多个变量的值。语句只有在守卫满足时才能执行 (即处理器此时被启用)。状态由所有变量的值定义。系统的状态 (又称为配置) 是所有处理器状态的笛卡儿积。我们用 Γ 表示系统的所有可能配置。

处理器 p 在配置 $\gamma \in \Gamma$ 中被启用, 当且仅当 γ 中至少有一条规则对 p 启用。设 P 是一组二元状态转换关系, 表示为 $\rightarrow$, 定义在 Γ 上。协议 P 的执行是一个配置序列:

$\epsilon = \gamma_0 \rightarrow \gamma_1 \rightarrow \dots$

1. Γ : 寄存器的值可以是任意的。
2. r_i : 两个相邻节点的寄存器值之间的差值小于或等于1 (这是一个局部的约束)。
3. r_i^* : 系统没有死锁 (存在一种“提升”机制, 可以重置当前的系统配置)。

关键点:

- r_i 是 r_i^* 的子集, r_i^* 是 r_i 的超集。满足更多约束的系统更接近无死锁的状态。

其中 $\forall i \geq 0, \gamma_i \rightarrow \gamma_{i+1} \in P$, 这称为一个步骤。如果 $\gamma_i = \gamma_{i+1}$, 则 γ_i 是一个终端配置。极大性意味着序列是有限的 (协议 P 在终端配置中停止) 或无限的。 E 是有可能执行的集合。如果处理器 p 在 γ_i 中被启用, 但在步骤 $\gamma_i \rightarrow \gamma_{i+1}$ 中未执行任何规则, 则称 p 在此步骤中被中和。

调度器 (也称为守护进程) 是执行的谓词。在任何执行中, 每个步骤 $\gamma \rightarrow \gamma'$ 都是通过从已启用处理器集中原子地选择并执行一条规则实现的。该选择由调度器控制。调度器可能具有以下属性:

- **中心调度器**: 在任何特定步骤中只选择一个已启用处理器。
- **分布式调度器**: 在任何步骤中选择至少一个已启用处理器。
- **局部中心调度器**: 在任何步骤中, 选择至少一个已启用处理器, 并确保两个邻居不会并发执行。
- **同步调度器**: 每一步选择所有已启用处理器。
- **异步调度器**: 如果是中心、分布式或局部中心的, 则称为异步调度器。

调度器也可能具有某些公平性属性:

- **强公平调度器**: 每个无穷次被启用的处理器最终都会被选择执行规则。
- **弱公平调度器**: 每个连续无穷次被启用的处理器最终会被选择执行规则。
- **不公平调度器**: 只保证至少有一个被启用的处理器最终会被选择执行规则。

强公平调度器是最强的公平假设, 任何在这种假设下无法解决的问题, 也无法在较弱的假设下解决。相比之下, 任何能在不公平调度器下执行的算法在更强的公平性假设下也能执行。

定义

我们考虑自稳定 (self-stabilizing) 算法, 即算法在有限时间内可以从任意数量的瞬态错误中恢复正确行为。以下是正式定义:

定义1 (自稳定性):

如果一个算法 A 对规范 S 满足以下两个性质, 则称其为自稳定的:

1. **闭包性 (Closure)**: 从任何满足 S 的配置开始, A 的任何执行始终满足 S 。
2. **收敛性 (Convergence)**: 从任何不满足 S 的配置开始, A 的任何执行在有限时间内达到满足 S 的配置。

1) 强公平 (Strongly fair)

定义 (人话 + 形式化)

- 人话: 只要某条规则**反复地**获得执行机会 (哪怕中间断断续续), 调度器**终有一天**会让它执行。
- 形式化: 如果一条规则在无限多步里是 *enabled* (enabled infinitely often), 则它最终会被执行一次。

时间线小例子

- R 在步骤: 1 ✓、3 ✓、5 ✓、7 ✓、... (偶尔可执行, 偶尔不可)
- 强公平 $\Rightarrow$ 不能永远跳过 R, 最终必须执行 R。

适用场景

- 解决“抖动/间歇可执行”的饥饿问题。
- 代价: 假设更强、现实系统里较难保证。

2) 弱公平 (Weakly fair)

定义 (人话 + 形式化)

- 人话: 如果从某个时刻开始, 某条规则**一直处于**可执行状态 (持续 *enabled*), 那它**终会被执行**。
- 形式化: 若存在时刻 t_0 , 使得 $\forall t \geq t_0$, 规则一直 *enabled* (continuously enabled), 则最终会被执行。

时间线小例子

- 从第 100 步开始, R 步步可执行 (100、101、102、... 都 ✓)。
- 弱公平 $\Rightarrow$ 不允许永远冷落 R, 必须执行它。
- 但如果 R 只是“断断续续可执行” (比如只在奇数步 ✓, 偶数步 ✗), **弱公平无法保证** R 会被执行; 这正是与强公平的差别。

适用场景

- 现实里更常见的假设: 只要不把某条**一直可执行**的规则饿死就行。

3) 不公平 (Unfair)

定义 (人话)

- 只保证“每一步**至少**有人在执行”, 但可能**永远**忽视某些一直可执行的规则。
- 饥饿 (starvation) 是允许发生的。

时间线小例子

- R 从 1 步开始一直 ✓, 但调度器每次都选别人。
- 不公平 $\Rightarrow$ 合法 (虽然很不厚道...)。

✗ + 2

2 令牌循环 (Token Circulation)

在本练习中，我们通过令牌循环 (Token Circulation) 来研究一个互斥协议。问题的规范如下：

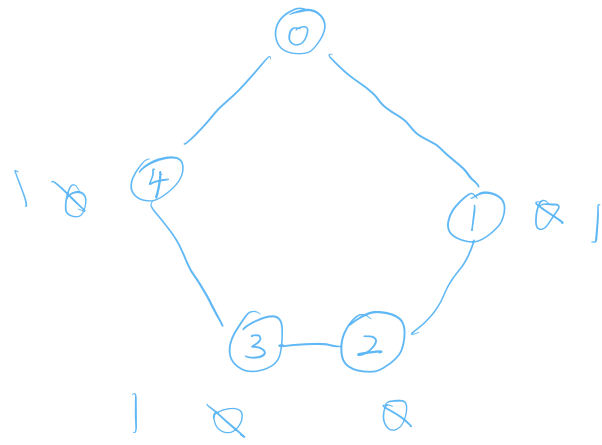
规范1 (互斥性, Mutual Exclusion)

- 活性 (Liveness)：任何处理器都能无限频繁地进入临界区。
- 安全性 (Safety)：在任何配置中，最多只有一个处理器处于临界区。

规范2 (令牌循环, Token Circulation)

- 活性 (Liveness)：任何处理器都能无限频繁地持有令牌。
- 安全性 (Safety)：在任何配置中，最多只有一个处理器持有令牌。

该网络是由 n 个处理器组成的环，记为 $p_0, p_1, \dots, p_{n-1}$ 。假设分布式不公平调度，每个处理器 p_i 只有一个变量 $c_i \in \{0, 1, \dots, k-1\}$ (其中 k 是一个满足 $k > n$ 的给定数字)。处理器 p_0 在 $c_0 = c_{n-1}$ 时持有令牌。处理器 p_i ($i \neq 0$) 在 $c_i \neq c_{i-1}$ 时持有令牌。协议如下：



问题：

1. 给出一个算法执行的例子，初始配置为： $n = 5$ ，且 $\forall i \in \{0, \dots, n-1\}, c_i = 0$ 。
2. 证明在任何配置中，至少有一个处理器持有令牌。
3. 证明从任何配置开始的任何执行中，任何处理器无限频繁地持有令牌。
4. 证明从仅存在一个令牌的配置开始的任何执行都满足令牌循环的规范。
5. 证明从任何配置开始的任何执行都会达到仅存在一个令牌的配置。
6. 总结。

算法 1：用于处理器 p_i 的自稳定令牌循环

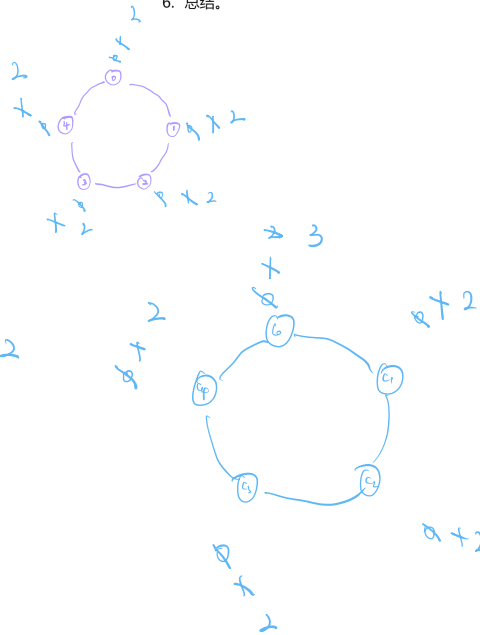
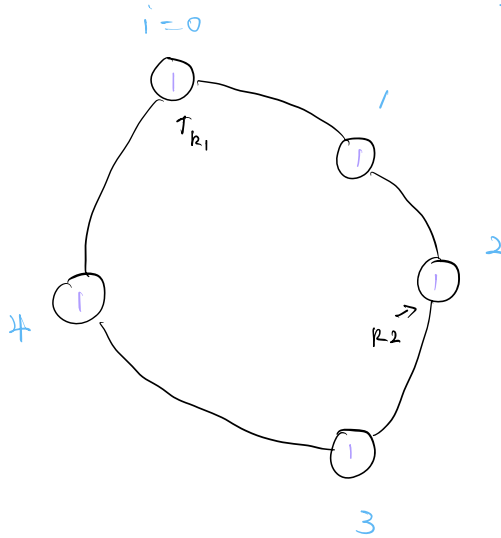
- 规则对于 $i = 0$ ：

$$(R_1): c_0 = c_{n-1} \rightarrow c_0 := c_0 + 1 \mod k$$

- 规则对于 $i \neq 0$ ：

$$(R_2): c_i \neq c_{i-1} \rightarrow c_i := c_{i-1}$$

Q1.



Q2. à pr. absurde

il ex it ue configuration dans la quelle il n'y a pas de jeton.

$$\Rightarrow \forall i \in \{1, \dots, n-1\}, c_i = c_{i+1}$$

$$\Rightarrow c_0 = c_1 = c_2 = \dots = c_{n-1}$$

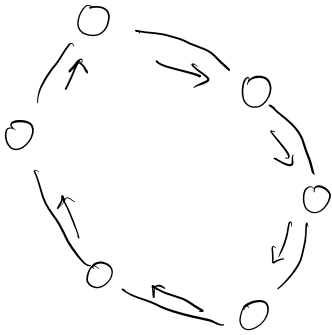
mais dans ce cas ,

p_0 peut appliquer R_1 contradiction

$\Rightarrow$ liveness : $\exists$ toujours 1 processus qui peut appliquer une règle $\neq \emptyset$ deadlock,

$\Rightarrow$ toute les exécution

$Q_2 + Q_3$: tout processus obtenir un jeton in finiment souvent.



翻译与解释

1. 反证法假设:

假设存在一种配置 (configuration) , 在这种配置下, 没有可用的令牌 (jeton)。

2. 逻辑推导:

- $\forall i \in \{1 \dots n-1\}, c_i = c_{i-1}$: 对于所有顶点 i , 其状态 c_i 等于前一个顶点的状态 c_{i-1} 。
- 推导出 $c_0 = c_1 = c_2 = \dots = c_{n-1}$ 。
即所有顶点状态相同。
- 但在这种情况下, 顶点 p_0 可以应用规则 R_1 , 这与「没有令牌」的假设矛盾 (Contradiction) 。

3. 结论:

• 活性 (Liveness):

始终存在至少一个进程能够应用某条规则 (即不存在死锁 $\emptyset$ deadlock) 。

1. 假设反例 (归谬法) :

- 假设存在一种配置, 其中没有任何处理器持有令牌。
- 根据令牌持有条件, 令牌仅在 $c_i \neq c_{i-1}$ 时被持有。
- 也就是说, 如果没有令牌, 就意味着对于所有 i ($i \in \{1, \dots, n-1\}$) , $c_i = c_{i-1}$ 。

2. 推出矛盾:

- 由于所有 c_i 值相同, 我们得到:

$$c_0 = c_1 = c_2 = \dots = c_{n-1}$$

- 但是, 在这种情况下, p_0 仍然遵循规则 R_1 :

$$c_0 := (c_0 + 1) \bmod k$$

- 这意味着** c_0 会发生变化** , 即:

$$c_0 \neq c_{n-1}$$

- 这样 p_0 仍然持有令牌, 从而导致假设“所有处理器都没有令牌”矛盾。

3. 结论 (反证法) :

- 由于假设“所有处理器都没有令牌”导致矛盾, 所以它不可能发生。
- 因此, 在任何配置中, 至少有一个处理器持有令牌。

Q3. Token $\equiv$ enable (activité)

QR $\Rightarrow \ominus$ 1 jeton

Absurde: $\exists$ une configuration γ_0 a une exécution

qui parte de γ_0 et un

processeur p_i tel que p_i

a le jeton un nombre

fini de fois

$\Rightarrow \exists \gamma_0, \gamma_1, \dots$

On se place la configuration γ_0 ,

p_i n'a plus jamais le jeton.

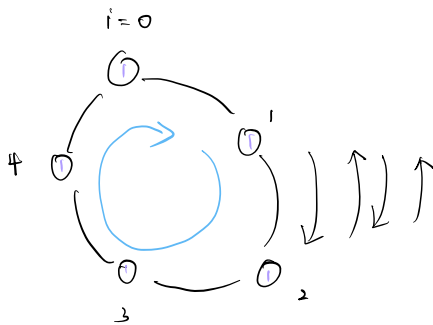
Par la Q2, on sait $\exists$ dans
toutes les configurations un marqueur.

问题:

1. 给出一个算法执行的例子, 初始配置为: $n = 5$, 且 $\forall i \in \{0, \dots, n-1\}, c_i = 0$.
2. 证明在任何配置中, 至少有一个处理器持有令牌。
3. 证明从任何配置开始的任何执行中, 任何处理器无限频繁地持有令牌。
4. 证明从仅存在一个令牌的配置开始的任何执行都满足令牌循环的规范。
5. 证明从任何配置开始的任何执行都会达到仅存在一个令牌的配置。
6. 总结。

Je preds

$\Rightarrow \exists j \in \{0, \dots, i-1\}$ tel que p_j a
infiniment
le jeton.



1. 令牌是激活条件 (Token = enable l'activité)

- 令牌的存在是处理器能够执行操作的关键条件。
- 问题要求证明 令牌不会永久滞留在某个处理器上，而是所有处理器都有机会持有令牌。

2. 归谬法 (Preuve par l'absurde) :

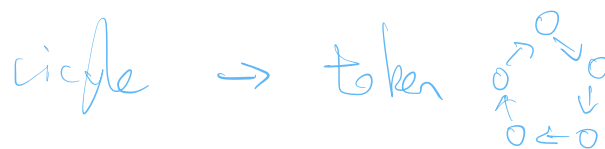
- 假设 存在一个处理器 p_i 仅持有有限次令牌，即 在某个配置 r_0 之后，它再也无法获得令牌。
- 这意味着 在所有后续配置中， p_i 不再持有令牌。

3. 推出矛盾:

- 根据第二问 (Q2)，系统中始终至少有一个处理器持有令牌 (即令牌不会消失)。
- 如果 p_i 再也无法持有令牌，那么一定存在某个处理器 p_j ($j \neq i$) 无限次持有令牌 (否则违反 Q2)。
- 但如果令牌无限次停留在某个固定的 p_j 上，那么令牌就不会继续传递，导致其他处理器无法持有令牌。
- 这违反了令牌循环 (Token Circulation) 的规则，因为令牌的设计本来就是让所有处理器都有机会持有。

4. 结论:

- 由于假设 p_i 只能持有有限次令牌导致矛盾，因此假设不成立。
- 也就是说，系统中的每个处理器 p_i 都会无限频繁地获得令牌。



如果节点无 token $\rightarrow$

token bloque 在节点前

$\Rightarrow$ deadlock.

1. 初始假设:

- 根据算法, 处理器 P_0 会无限次注入一个新值, 确保某个处理器 P_i 能无限次执行规则 R_2 。
- 因此, 处理器 P_0 也需要无限次执行规则 R_1 。
- 在任意时间 $k \in [0, t]$, 处理器 P_i 必须能够无限次获得令牌。

3. 矛盾:

- 然而, 假设处理器 P_{n-1} 在某个配置 γ_{k0} 之后被阻塞, 即它再也无法获得令牌。
- 这与 P_i 必须无限次获得令牌的假设相矛盾。

2. 推导逻辑:

- 假设 P_0 最终注入了一个值 d , 而 d 最初在网络中是不存在的。
- 为了 P_0 能注入 d , 它必须无限次执行规则 R_1 。
- 结果是: 状态 C_{n-1} 和 C_0 在某些时间点会无限次相等 (系统周期性循环)。

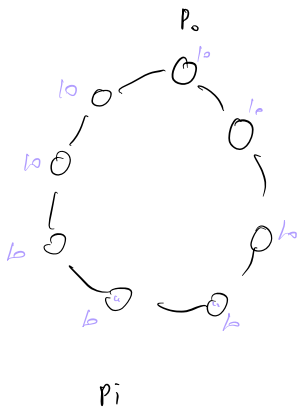
4. 结论:

由于矛盾的存在, 证明了初始假设是错误的。即, 系统中:

- 每个处理器都会定期获得令牌;
- 系统中不存在死锁或阻塞情况。

Q4. En partant d'une configuration dans laquelle, il n'y a qu'un seul jeton, alors tous les processus ont le jeton à l'infini, et dans toute configuration, il y a au plus 1 jeton 'safety'.

Q2 et Q3 : tout processus obtient 1 jeton infiniment souvent.



$$C_{i+1} \neq C$$

$$\text{et } \forall k \in \{0 \dots k-1\},$$

$$C_k = C_{i-1}$$

1. Q4 的定义与假设：

- 从一个初始配置出发，在这个配置中，系统中只有一个令牌。
- 结论：
 - 活性 (Liveness)：所有进程都能无限次获得令牌。
 - 安全性 (Safety)：在任意时刻，整个系统中最多有一个令牌（保证系统的安全性）。

2. 结合 Q2 和 Q3：

- Q2 和 Q3 总结：

所有进程在执行过程中都会无限次获得至少一个令牌。

问题：

- 给出一个算法执行的例子，初始配置为： $n = 5$ ，且 $\forall i \in \{0, \dots, n - 1\}, c_i = 0$ 。
- 证明在任何配置中，至少有一个处理器持有令牌。
- 证明从任何配置开始的任何执行中，任何处理器无限频繁地持有令牌。
- 证明从仅存在一个令牌的配置开始的任何执行都满足令牌循环的规范。
- 证明从任何配置开始的任何执行都会达到仅存在一个令牌的配置。
- 总结。

1. 令牌循环的基本假设

- 令牌的传递规则：
 - 处理器 p_0 在 $c_0 = c_{n-1}$ 时持有令牌，然后执行：

$$c_0 := (c_0 + 1) \mod k$$

- 其余的处理器 p_i 在 $c_i \neq c_{i-1}$ 时持有令牌，然后执行：

$$c_i := c_{i-1}$$

- 这意味着令牌的存在条件是 $c_i \neq c_{i-1}$ 。

2. 归谬法 (Proof by Contradiction)

假设系统中存在某个时刻有两个处理器同时持有令牌，即存在 p_i 和 p_j ($i \neq j$)，它们都满足 $c_i \neq c_{i-1}$ 。

- 这意味着 p_i 和 p_j 都有令牌。
- 但根据令牌的传递方式，令牌应该按照环形顺序唯一地传递，一个处理器只有在前一个处理器释放令牌后才能持有令牌。
- 因此，如果同时有两个处理器持有令牌，说明：
$$c_i \neq c_{i-1} \quad \text{且} \quad c_j \neq c_{j-1}$$
但这意味着在两个独立的地方都发生了令牌的转移，这是不可能的，因为令牌是唯一的。
- 这与令牌的传递规则矛盾，因此假设不成立。

3. 结合 Q2 和 Q3 的结论

- Q2 证明：始终至少有一个处理器持有令牌（令牌不会丢失）。
- Q3 证明：所有处理器都能无限次获得令牌（不会死锁）。
- 结合以上两个结论，我们可以进一步说明：
 - 令牌既不会消失，也不会滞留在某个处理器上，它必须不断传递。
 - 由于令牌只能从一个处理器传递到另一个处理器，整个系统中始终最多只有一个令牌存在。

4. 结论

- 在任何时刻，系统中最多只有一个处理器持有令牌。
- 这保证了令牌循环的安全性 (Safety)，即：

$$\forall t, \quad \sum_{i=0}^{n-1} 1(p_i \text{ 持有令牌}) \leq 1$$

其中 $1(\cdot)$ 是指示函数，表示某个处理器是否持有令牌。

- 因此，令牌循环协议满足安全性要求，即不会出现多个令牌同时存在的情况。

Q 5.

Puisque pour les questions Q2, Q3, tous problèmes existe sa règle en suivant.

1. 背景:

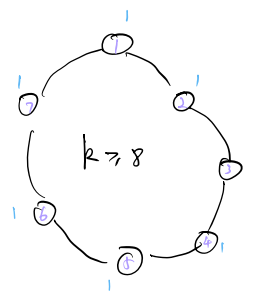
- 根据 Q2 和 Q3 的推导, 所有处理器的规则都会被无限次执行。

2. 推导过程:

- 假设 $k > n$ (处理器的编号超出了某一阈值)。
- 在一个有限的时间内 (最坏情况下为 n^3 个步骤), 处理器 P_0 会注入一个新值 d , 而 d 最初在网络中并不存在。
- 经过 n 个步骤后, 所有处理器的状态 $C_i = d$ 。
- 在此配置下, 系统中仅保留一个令牌。

3. 结论:

- 系统在某个时间点会收敛到一个状态 (所有处理器的状态一致), 并且系统内始终保持单一令牌的安全性。



解答的逻辑

1. 依据 Q2 和 Q3

- Q2 (活性 - Liveness)** 证明了: 所有处理器都会无限频繁地持有令牌, 令牌不会丢失。
- Q3 (活性 - Liveness)** 进一步证明了: 所有处理器都会周期性地持有令牌, 即令牌会在环形结构中传递, 而不会停滞在某个处理器中。

这两个结论表明:

- 系统不会死锁 (因为令牌不断流动)。
- 令牌不会永远滞留在某个处理器上 (所有处理器都能获取令牌)。

2. 证明系统会收敛到最多一个令牌的状态

假设初始时刻系统中可能存在多个令牌, 我们要证明:

在有限时间后, 系统会演化到一个最多只含有一个令牌的状态。

(1) 假设 $k > n$

- 这里的 $k > n$ 表示处理器的状态取值范围比处理器数量 n 还大, 意味着状态变量 c_i 的编码空间足够大, 能够确保所有处理器最终能达到相同的状态。
- 设系统最开始处于某个状态 S_0 , 即各个处理器的状态可能不一致, 甚至可能有多个令牌存在。

(2) 令牌的传播使系统状态逐步趋于一致

- 由于 Q3 证明了所有处理器都会无限次持有令牌, 那么每个处理器都会不断调整自身的状态, 使得:

$$c_i := c_{i-1}$$

这表示处理器的状态将逐渐趋于统一。

- 进一步, 考虑最坏情况:

- 最坏情况下, 可能需要 n^3 步来使整个系统状态完全统一 (其中 n^3 只是一个上界, 用于保证在有限时间内完成)。
- 但无论如何, 状态传递的规则保证了所有 c_i 最终都会趋于相同的值 d , 即:

$$c_0 = c_1 = c_2 = \dots = c_{n-1} = d$$

- 这意味着整个系统达到了一个收敛状态。

问题:

- 给出一个算法执行的例子, 初始配置为: $n = 5$, 且 $\forall i \in \{0, \dots, n-1\}, c_i = 0$ 。
- 证明在任何配置中, 至少有一个处理器持有令牌。
- 证明从任何配置开始的任何执行中, 任何处理器无限频繁地持有令牌。
- 证明从仅存在一个令牌的配置开始的任何执行都满足令牌循环的规范。
- 证明从任何配置开始的任何执行都会达到仅存在一个令牌的配置。
- 总结。

(3) 最终状态中只有一个令牌

- 在收敛状态中, 所有处理器的状态变量相同 $c_0 = c_1 = \dots = c_{n-1}$ 。
- 根据令牌的持有条件:

$$c_i \neq c_{i-1}$$

- 但如果所有 c_i 都相等, 则无法满足 $c_i \neq c_{i-1}$, 即没有处理器持有多余的令牌。
- 只有 p_0 通过规则 R_1 改变 c_0 , 从而使得新的令牌在系统中继续传递, 但最多仍然只有一个令牌存在。

3. 结论

- 经过有限步的传播, 所有处理器的状态最终收敛到一个稳定状态, 即:

$$c_0 = c_1 = \dots = c_{n-1}$$

- 在这个状态下, 系统保证:
 - 只有一个令牌存在 (满足安全性)。
 - 令牌仍然在循环中流动 (满足活性)。
- 最终, 系统将处于一个最多只有一个令牌的配置, 并保持令牌循环机制。

$Q_2, Q_3 \text{ et } Q_5 \Rightarrow \text{convergence}$ 收敛

$Q_2, Q_3, \text{ et } Q_5 \Rightarrow \text{cl\^oture}$ 闭包

1. Q2、Q3 和 Q5 的总结:

- **Convergence (收敛性) :**

- 通过 Q2 (所有进程无限次获得令牌)、Q3 (规则的执行无限次发生) 和 Q5 的性质, 可以证明系统状态会收敛到一致性配置。

- **Cl\^oture (闭包性) :**

- 通过 Q2、Q3 和 Q5, 可以证明系统无论从哪个初始状态开始, 都能够保持安全性和活性 (闭包性保障) 。

定义

我们考虑 **自稳定** (self-stabilizing) 算法, 即 算法在有限时间内可以从任意数量的瞬态错误中恢复正确行为。以下是正式定义:

定义1 (自稳定性) :

如果一个算法 A 对规范 S 满足以下两个性质, 则称其为自稳定的:

1. **闭包性 (Closure) :** 从任何满足 S 的配置开始, A 的任何执行始终满足 S 。
2. **收敛性 (Convergence) :** 从任何不满足 S 的配置开始, A 的任何执行在有限时间内达到满足 S 的配置。

3. Maximal Matching

3 最大匹配 (Maximal Matching)

在本练习中，我们研究最大匹配的问题。问题的规范如下：

定义2 (匹配, Matching)

给定一个图 $G = (V, E)$ ，图上的一个匹配 M 是 E 的一个子集，使得 V 中的任何节点至多属于 M 中的一条边。如果不存在匹配 M' ，满足 $M \subset M'$ ，则称 M 为最大匹配。

规范3 (最大匹配, Maximal Matching)

- **活性 (Liveness)**：协议在有限时间内终止。
- **安全性 (Safety)**：在终端配置中，系统存在一个最大匹配。

- 设 $V = \{A, B, C, D\}$ ， $E = \{(A, B), (B, C), (C, D)\}$ 。
- 可能的匹配集：
 - $M_0 = \{(A, B), (C, D)\}$ (最大匹配)
 - $M_1 = \{(B, C)\}$ (可以扩展)
 - $M_2 = \{(A, B), (B, C)\}$ (冗余)

网络是一个任意图。假设中心不公平调度，每个处理器 p 有一个变量 pref_p ，其值属于 $N_p \cup \{\text{null}\}$ 。此变量指代 p 的首选邻居，用于匹配。例如，如果 $\text{pref}_p = q$ ，那么 p 希望将边 $\{p, q\}$ 加入匹配中。对于任意处理器 p ，我们定义以下谓词：

- waiting_p : $(\text{pref}_p = q) \wedge (\text{pref}_q = \text{null})$
- matched_p : $(\text{pref}_p = q) \wedge (\text{pref}_q = p)$
- chaining_p : $(\text{pref}_p = q) \wedge (\text{pref}_q = r) \wedge (r \neq p)$
- dead_p : $(\text{pref}_p = \text{null}) \wedge (\forall q \in N_p, \text{matched}(q) = \text{true})$
- free_p : $(\text{pref}_p = \text{null}) \wedge (\exists q \in N_p, \text{matched}(q) = \text{false})$

- **waiting_p** (等待) : $(\text{pref}_p = q) \wedge (\text{pref}_q = \text{null})$
 - p 选择了 q 但 q 还未匹配。
- **matched_p** (匹配成功) : $(\text{pref}_p = q) \wedge (\text{pref}_q = p)$
 - p 和 q 彼此选择，形成匹配。
- **chaining_p** (链式匹配) : $(\text{pref}_p = q) \wedge (\text{pref}_q = r) \wedge (r \neq p)$
 - p 选择了 q ，但 q 选择了其他节点 r 。
- **dead_p** (无匹配) : $(\text{pref}_p = \text{null}) \wedge (\forall q \in N_p, \text{matched}(q) = \text{true})$
 - p 没有匹配，并且所有邻居都已匹配。
- **free_p** (自由) : $(\text{pref}_p = \text{null}) \wedge (\exists q \in N_p, \text{matched}(q) = \text{false})$
 - p 还没有匹配，但邻居中至少有一个也未匹配。

很容易验证，对于任意配置 γ 和任意处理器 p ，上述谓词中恰好有一个对 p 为真。

中心不公平调度 (Centrally Unfair Scheduling)

中心不公平调度 (Centrally Unfair Scheduling) 是一种调度策略。在这种策略中，调度决策是由一个中心化的调度器控制的，而调度的方式可能是不公平的，即：

1. 某些进程或任务可能比其他任务获得更多的执行机会。
2. 调度策略可能会偏向某些特定的任务或资源分配模式。

在分布式系统或者多进程调度中，“不公平”通常意味着：

- 并非所有进程或任务都能均等地获得 CPU 资源、I/O 访问或其他计算资源。
- 可能会导致某些进程长期得不到调度 (饥饿问题, Starvation)。
- 优先级较高的任务可能会频繁执行，而低优先级的任务被推迟甚至无限期延迟。

2.1 匹配规则 (Matching Rule, M)

$(M) : (\text{pref}_p = \text{null}) \wedge (\exists q \in N_p, \text{pref}_q = p) \rightarrow \text{pref}_p := q$

- 当 p 未匹配 ($\text{pref}_p = \text{null}$)，并且有邻居 q 想要匹配 p ($\text{pref}_q = p$)，那么 p 选择 q 进行匹配。

2.2 引诱规则 (Seduction Rule, S)

$(S) : (\text{pref}_p = \text{null}) \wedge (\forall q \in N_p, \text{pref}_q \neq p) \wedge (\exists q \in N_p, \text{pref}_q = \text{null}) \rightarrow \text{pref}_p := q$

- 当 p 仍然未匹配 ($\text{pref}_p = \text{null}$)，并且：

1. 没有邻居 q 选择 p 作为匹配对象 ($\forall q \in N_p, \text{pref}_q \neq p$)
2. 存在至少一个邻居 q 也未匹配 ($\exists q \in N_p, \text{pref}_q = \text{null}$)
3. 那么 p 选择 q 进行匹配 ($\text{pref}_p := q$)。

2.3 放弃规则 (Abandonment Rule, A)

$(A) : (\text{pref}_p = q) \wedge (\text{pref}_q \neq p) \wedge (\text{pref}_q \neq \text{null}) \rightarrow \text{pref}_p := \text{null}$

- 当 p 选择了 q 作为匹配对象 ($\text{pref}_p = q$)，但 q 没有选择 p ($\text{pref}_q \neq p$)，或者 q 已经匹配了其他节点，那么 p 放弃匹配，设为 null 。

算法2：用于处理器 p 的自稳定最大匹配

规则：

1. 匹配规则 (Matching Rule) :

$(M) : (\text{pref}_p = \text{null}) \wedge (\exists q \in N_p, \text{pref}_q = p) \rightarrow \text{pref}_p := q$

2. 引诱规则 (Seduction Rule) :

$(S) : (\text{pref}_p = \text{null}) \wedge (\forall q \in N_p, \text{pref}_q \neq p) \wedge (\exists q \in N_p, \text{pref}_q = \text{null}) \rightarrow \text{pref}_p := q$

3. 放弃规则 (Abandonment Rule) :

$(A) : (\text{pref}_p = q) \wedge (\text{pref}_q \neq p) \wedge (\text{pref}_q \neq \text{null}) \rightarrow \text{pref}_p := \text{null}$

问题：

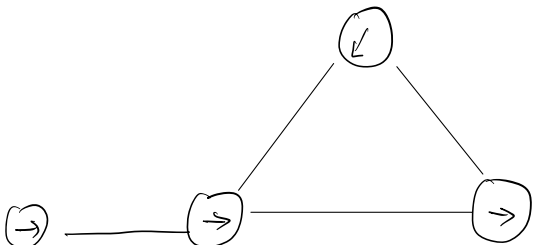
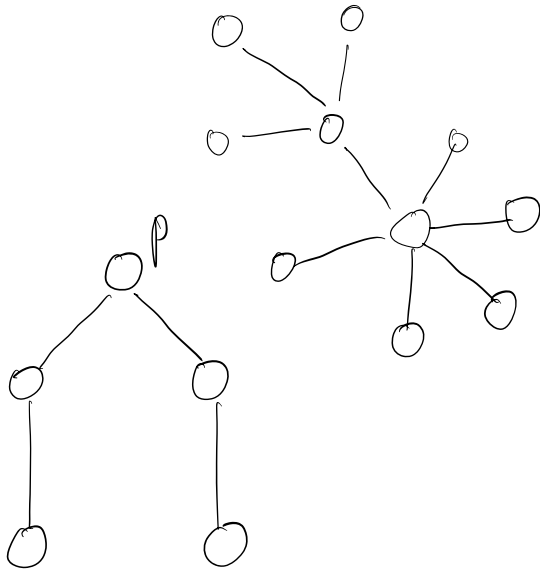
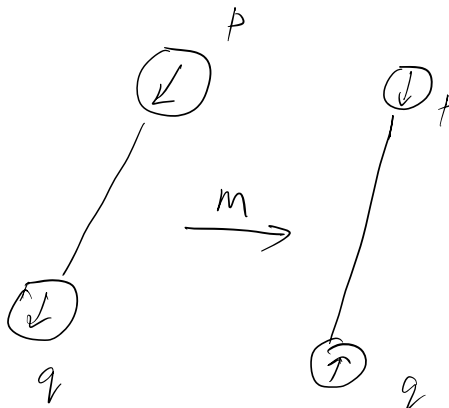
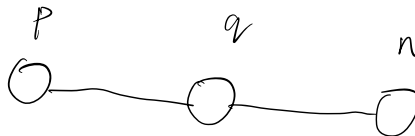
1. 证明交集集合 $\{\{p, \text{pref}_p\} | \text{pref}_p \neq \text{null}\}$ 在满足 $\forall p \in V, \text{matched}_p \vee \text{dead}_p$ 的配置中是一个最大匹配。
2. 证明任何终端配置都满足 $\forall p \in V, \text{matched}_p \vee \text{dead}_p$ 。
3. 证明任何满足 $\forall p \in V, \text{matched}_p \vee \text{dead}_p$ 的配置是一个终端配置。
4. 证明从任何配置开始的任何执行都能在有限时间内达到一个终端配置。提示：考虑等待处理器的数量 w 、链式处理器的数量 c 和空闲处理器的数量 f 。证明函数

$$P : \gamma \in \Gamma \rightarrow (w + c + f, 2c + f)$$

在任何执行中是严格递减的 (使用字典序顺序)。

5. 总结。

$$I \leq NPI$$



Q1

2. 证明思路

我们需要证明：

- (1) 这个匹配是合法的（即 没有节点属于多条匹配边）。
- (2) 这个匹配是最大的（即 不能再添加新的边）。

(1) 匹配是合法的

考虑边集合：

$$M = \{ \{p, \text{pref}_p\} | \text{pref}_p \neq \text{null} \}$$

- 由于 匹配规则（Matching Rule, M）保证：

$$\text{pref}_p = q \quad \text{当且仅当} \quad \text{pref}_q = p$$

这意味着每个匹配边 $\{p, q\}$ 是双向同意的。

- 不存在某个节点属于两条不同的匹配边：

- 如果 p 选择了 q ($\text{pref}_p = q$) , 那么 p 不能再选择其他节点。
- 由于 $\text{pref}_q = p$, 这意味着 q 也不能再选择其他节点。
- 因此, 每个节点最多属于一条匹配边, 保证了匹配的合法性。

(2) 匹配是最大的

- 假设匹配不是最大匹配, 则应该存在某些未匹配的节点 p 和 q , 它们可以形成新的匹配边 $\{p, q\}$ 。
- 但题目已给定 $\forall p \in V, \text{matched}_p \vee \text{dead}_p$, 即:
 - 所有节点要么已匹配, 要么无法匹配。
- 这意味着:
 - 如果 p 是未匹配的 ($\text{pref}_p = \text{null}$) , 那么它的所有邻居 q 都已经匹配 (否则它会尝试匹配) 。
 - 因此, 不会存在两个都未匹配的节点 p, q 可以形成新的匹配边。
- 这与“匹配不是最大匹配”矛盾, 所以假设不成立。

因此, 匹配是最大的。

Q2

2. 证明思路

我们利用算法的匹配规则 (M)、引诱规则 (S)、放弃规则 (A), 来证明所有节点最终都会进入 matched 或 dead 状态。

(1) 如果 p 处于 matched 状态

- 由 匹配规则 (M), 匹配关系是双向的:
 - 如果 p 选择了 q ($\text{pref}_p = q$) , 那么 q 必须也选择了 p ($\text{pref}_q = p$) 。
 - 即 $\{p, q\}$ 形成一个匹配边, 且不会再改变。
- 这样, p 和 q 进入 matched 状态, 并且不会被改变。

(2) 如果 p 仍然未匹配

考虑 p 仍然未匹配的情况 (即 $\text{pref}_p = \text{null}$) 。

- 由 引诱规则 (S) :
 - 如果 p 仍然未匹配, 且存在邻居 q 也未匹配, 那么 p 会选择 q 作为匹配对象。
 - 由于规则 S 只要符合条件就会执行, 因此如果 p 仍然未匹配, 说明所有邻居 q 已经匹配 (否则 p 早就执行 S 规则) 。
- 因此, p 进入 dead 状态, 即:

$$\text{dead}_p = (\text{pref}_p = \text{null}) \wedge (\forall q \in N_p, \text{matched}_q = \text{true})$$

- 这意味着 p 无法匹配, 因为所有邻居都已匹配。

(3) 如果 p 处于 chaining 状态

- p 选择了 q , 但 q 选择了另一个 r ($\text{pref}_q = r$) 。
- 由 放弃规则 (A) , p 会执行:

$$\text{pref}_p := \text{null}$$

- 这样, p 进入未匹配状态, 并重新尝试匹配 (执行 S 规则) 。
- 根据 (2) , p 最终要么匹配成功, 要么进入 dead 状态。

3. 归纳推理

- 初始状态: 节点 p 可能是自由状态 (free) , 即 $\text{pref}_p = \text{null}$ 。
- 算法执行过程中:
 - 如果 p 有未匹配的邻居, 它会尝试匹配 (执行 S 规则) 。
 - 如果所有邻居已经匹配, 它不会再改变 (进入 dead_p 状态) 。
 - 如果 p 处于 chaining 状态, 它最终会放弃当前匹配, 并重新尝试, 直到匹配或进入 dead 状态。
- 终端状态:
 - 所有未匹配的节点 p 必须满足 $\forall q \in N_p, \text{matched}_q = \text{true}$, 即进入 **dead** 状态。
 - 其他节点都是 **matched**。

因此, 最终所有节点都满足 $\text{matched}_p \vee \text{dead}_p$, 证毕。

Q3

2. 证明思路

要证明该配置是终端配置, 我们需要逐步分析三个匹配规则 (M、S、A), 并证明这些规则在该配置下都不会再执行。

(1) 匹配规则 (M) 不会再次执行

匹配规则 (M) 如下:

$$(M) : (\text{pref}_p = \text{null}) \wedge (\exists q \in N_p, \text{pref}_q = p) \rightarrow \text{pref}_p := q$$

- 该规则的触发条件是:
 - p 未匹配 ($\text{pref}_p = \text{null}$) 。
 - 存在某个邻居 q 选择了 p ($\text{pref}_q = p$) 。

但题目已假设 $\forall p \in V, \text{matched}_p \vee \text{dead}_p$:

- 对于 **matched** 节点: 它已经匹配, 不可能触发 (M) 规则。
- 对于 **dead** 节点:
 - $\text{pref}_p = \text{null}$, 但其所有邻居 q 已经匹配 (否则它不可能是 **dead**) 。
 - 这意味着 **不会存在 q 选择 p** (即 $\forall q, \text{pref}_q \neq p$) 。
- 因此 匹配规则 M 不会触发。

✔ 匹配规则 (M) 在该配置下不会再次执行。

(2) 引诱规则 (S) 不会再次执行

引诱规则 (S) 如下:

$$(S) : (\text{pref}_p = \text{null}) \wedge (\forall q \in N_p, \text{pref}_q \neq p) \wedge (\exists q \in N_p, \text{pref}_q = \text{null}) \rightarrow \text{pref}_p := q$$

- 该规则的触发条件是:
 - p 仍然未匹配 ($\text{pref}_p = \text{null}$) 。
 - 所有邻居 q 没有选择 p 作为匹配 ($\forall q, \text{pref}_q \neq p$) 。
 - 至少存在一个邻居 q 也是未匹配的 ($\exists q, \text{pref}_q = \text{null}$) 。

但题目已假设 $\forall p \in V, \text{matched}_p \vee \text{dead}_p$:

- 对于 **matched** 节点, 它不会执行 (S) 规则。
- 对于 **dead** 节点:
 - 它的所有邻居 q 已经匹配 (否则它不会进入 **dead** 状态) 。
 - 因此不会存在 q 是未匹配的 (矛盾) 。
 - 这意味着 (S) 规则的触发条件不满足, 所以不会执行。

✔ 引诱规则 (S) 在该配置下不会再次执行。

(3) 放弃规则 (A) 不会再次执行

放弃规则 (A) 如下:

$$(A) : (\text{pref}_p = q) \wedge (\text{pref}_q \neq p) \wedge (\text{pref}_q \neq \text{null}) \rightarrow \text{pref}_p := \text{null}$$

- 该规则的触发条件是:
 - p 选择了 q ($\text{pref}_p = q$) 。
 - q 没有选择 p ($\text{pref}_q \neq p$) 。
 - q 选择了其他节点 r ($\text{pref}_q \neq \text{null}$) 。

但题目已假设 $\forall p \in V, \text{matched}_p \vee \text{dead}_p$:

- 对于 **matched** 节点:
 - 匹配关系是双向的 ($\text{pref}_p = q$ 且 $\text{pref}_q = p$) , 不会满足 $\text{pref}_q \neq p$ 。
 - 所以不会触发 (A)。
- 对于 **dead** 节点:
 - 它已经是 $\text{pref}_p = \text{null}$, 所以不会触发 (A)。

✔ 放弃规则 (A) 在该配置下不会再次执行。

4. 结论

- 我们证明了在该配置下:
 - 匹配规则 M 不会触发 (所有未匹配的节点都无法匹配) 。
 - 引诱规则 S 不会触发 (所有未匹配的节点的邻居都已匹配) 。
 - 放弃规则 A 不会触发 (所有匹配关系都是稳定的) 。
- 由于没有规则可以再次执行, 该配置就是终端配置。

✔ 结论: 所有满足 $\forall p \in V, \text{matched}_p \vee \text{dead}_p$ 的配置, 都是终端配置。

1. 题意解析

Q4.

这道题的目标是 证明算法在有限步内必然收敛到一个终端配置，即：

- 不会无限执行下去（避免死循环或死锁）。
- 最终系统达到稳定状态（终端配置）。

关键概念

- 等待处理器（waiting processor, w ）：
 - 这些节点已经选择了一个匹配对象 q （即 $\text{pref}_p = q$ ），但 q 还未选择 p 。
- 链式处理器（chaining processor, c ）：
 - 这些节点选择了 q （ $\text{pref}_p = q$ ），但 q 选择了 r （ $\text{pref}_q = r$ ），且 $r \neq p$ 。
- 空闲处理器（free processor, f ）：
 - 这些节点的 $\text{pref}_p = \text{null}$ ，并且仍然有未匹配的邻居。

2. 证明思路

为了证明有限时间内达到终端配置，我们需要构造一个严格递减的势能函数 $P(\gamma)$ ，该函数在执行规则 M （匹配）、 S （引诱）和 A （放弃）时严格递减。这就保证了每一步执行都会让系统更接近终端状态，最终算法在有限时间内终止。

3. 定义势能函数

题目提示我们使用以下势能函数：

$$P(\gamma) = (w + c + f, 2c + f)$$

- 其中，第一个数值 $w + c + f$ 是**所有非终端处理器（等待、链式、自由处理器）**的数量。
- 第二个数值 $2c + f$ 进一步量化了链式和自由处理器的情况。

目标是证明这个函数在执行算法步骤后总是递减。

4. 规则如何影响 $P(\gamma)$

我们依次分析三条规则（匹配 M 、引诱 S 、放弃 A ），看它们如何影响势能函数：

(1) 匹配规则 M 作用

$$(M) : (\text{pref}_p = \text{null}) \wedge (\exists q \in N_p, \text{pref}_q = p) \rightarrow \text{pref}_p := q$$

- 这条规则的作用是：等待的处理器 p 变成匹配状态（matched）。
- 影响：
 - 减少 w （等待处理器的数量）。
 - 不影响 c （链式处理器的数量）。
 - 不影响 f （自由处理器的数量）。

所以 $P(\gamma) = (w + c + f, 2c + f)$ 的第一个分量减少，整体递减。

(2) 引诱规则 S 作用

$$(S) : (\text{pref}_p = \text{null}) \wedge (\forall q \in N_p, \text{pref}_q \neq p) \wedge (\exists q \in N_p, \text{pref}_q = \text{null}) \rightarrow \text{pref}_p := q$$

- 这条规则的作用是：自由的处理器 p 选择一个未匹配的邻居 q 进行匹配。
- 影响：
 - 减少 f （自由处理器的数量减少）。
 - 增加 w （等待处理器的数量增加，因为 p 选择了 q ，但 q 还没有选择 p ）。

f 减少，使得 $P(\gamma)$ 在字典序下递减，确保整体收敛。

(3) 放弃规则 A 作用

$$(A) : (\text{pref}_p = q) \wedge (\text{pref}_q \neq p) \wedge (\text{pref}_q \neq \text{null}) \rightarrow \text{pref}_p := \text{null}$$

- 这条规则的作用是：当 p 选择的 q 没有选择 p 时， p 放弃匹配，变回自由状态。
- 影响：
 - 减少 c （链式处理器的数量减少）。
 - 增加 f （空闲处理器的数量增加）。

$2c + f$ 递减，因此 $P(\gamma)$ 仍然递减，确保最终收敛。

5. 证明有限时间收敛

由于：

- 每次执行规则 M, S, A 都使得 $P(\gamma)$ 严格递减。
- 势能函数 $P(\gamma)$ 只有有限多种取值（因为 w, c, f 的取值是有限的）。
- 因此，系统在有限步内必须达到一个无法再递减的状态，这就是终端配置。

结论：所有执行都能在有限时间内达到终端配置，证明完成。

6. 总结

(1) 势能函数 $P(\gamma) = (w + c + f, 2c + f)$ 在算法执行过程中严格递减

- 匹配规则 M 让等待处理器减少，使 $P(\gamma)$ 递减。
- 引诱规则 S 让自由处理器减少，使 $P(\gamma)$ 递减。
- 放弃规则 A 让链式处理器减少，使 $P(\gamma)$ 递减。

(2) 由于 $P(\gamma)$ 只有有限种可能值，因此系统在有限步内必须停止

- 终端配置（所有处理器 $\text{matched}_p \vee \text{dead}_p$ ）是唯一可能的稳定状态。

(3) 结论

从任何初始配置开始，系统都能在有限步内终止于一个终端配置。

Q 5

1. 题目要求

第五题是前面所有证明的总结，我们需要综合之前的结论，说明：

1. 算法最终会收敛（**有限时间内终止**）。
2. 终端配置是**最大匹配（Maximal Matching）**。
3. 整个算法满足**活性（Liveness）**和**安全性（Safety）**。

2. 结合前面四道题的结论

我们已经通过前四题证明了以下核心性质：

题号	结论
第一题	终端状态中的匹配是一个 最大匹配 。
第二题	终端状态中所有节点都满足 $\text{matched}_p \vee \text{dead}_p$ （要么匹配，要么无法匹配）。
第三题	任何满足 $\text{matched}_p \vee \text{dead}_p$ 的配置，都是终端配置（不会再生状态变化）。
第四题	算法在有限步内必然收敛到 终端配置 （通过构造严格递减的势能函数 $P(\gamma)$ 证明）。

3. 终端状态是最大匹配

从**第一题**的结论，我们知道：

$$M = \{\{p, \text{pref}_p\} | \text{pref}_p \neq \text{null}\}$$

是**最大匹配**，因为：

- **合法性**：没有节点属于多条匹配边（匹配是双向的）。
- **最大性**：所有未匹配的节点都无法再匹配（它们的邻居已经匹配），因此无法再扩展匹配。

因此，算法保证了： ☒ **最终匹配是最大匹配（Maximal Matching）**。

4. 终端状态是唯一稳定状态

从**第二题**和**第三题**的结论：

- **第二题** 证明了**所有终端配置都满足 $\text{matched}_p \vee \text{dead}_p$** 。
- **第三题** 证明了**任何满足 $\text{matched}_p \vee \text{dead}_p$ 的配置都是终端配置**。

因此： ☒ **系统最终只能收敛到一个最大匹配状态，并且不会发生状态波动。**

5. 算法能在有限步内终止

从**第四题**的结论：

- 构造势能函数 $P(\gamma) = (w + c + f, 2c + f)$ ，并证明：
 - 每次执行规则（M、S、A）都会严格递减 $P(\gamma)$ 。
 - 由于 $P(\gamma)$ 只能取有限个值，算法必须在有限步内终止。

因此： ☒ **算法不会进入无限循环，必然在有限步内终止（满足活性 Liveness）**。

6. 结论总结

(1) 活性（Liveness）

- 算法保证终止（第四题）。
- 系统最终会达到一个终端配置（第三题）。
- 所有节点都会匹配或确定自己无法匹配（第二题）。

☒ 算法在有限步内完成，不会无限运行。

(2) 安全性（Safety）

- 最终状态是最大匹配（第一题）。
- 匹配是合法的（每个节点最多属于一条匹配边）。
- 不能再增加新的匹配边（因为所有未匹配节点都无法匹配）。

☒ 最终状态中的匹配是合法的最大匹配，保证安全性。

7. 最终结论

该算法是正确的，并且满足所有要求：

1. 在有限步内终止（Liveness）。
2. 终端状态是最大匹配（Maximal Matching），不能再扩展（Safety）。
3. 任何执行都能达到终端配置，并且终端配置是唯一稳定的状态。

☒ 该分布式算法能够正确、高效地求解最大匹配问题。

exam.

démon central.

中央守护进程？

1 seul nœud à la fois

démon distribué

分布式守护进程

plusieurs nœuds à la fois

- asynchrone

1. 中心守护进程 vs. 分布式守护进程

- 中心守护进程 (Démon Central) :

- 一次只有一个节点可以执行操作。
- 需要某种中心化的控制方式 (类似于集中式调度器)。
- 可能导致瓶颈 (bottleneck) 问题, 因为所有任务都必须等待中央控制节点。

- 分布式守护进程 (Démon Distribué) :

- 多个节点可以同时执行操作, 通常是异步的 (asynchrone)。
- 可以提高并行度, 但需要协调机制以避免冲突 (例如使用分布式锁、消息传递、共识算法等)。
- 可能导致不公平 (unfairness) 的问题, 某些节点可能一直得不到执行机会。

garde $\rightarrow$ action.

for (i, i,) {

si garde alors Action faire.

}

2. 代码和示意图

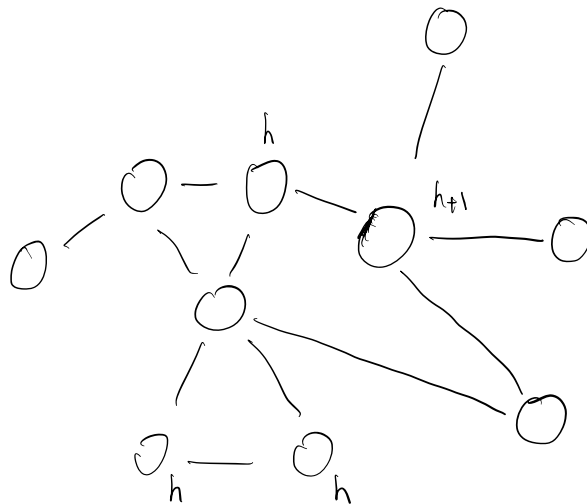
- 代码部分：

cpp

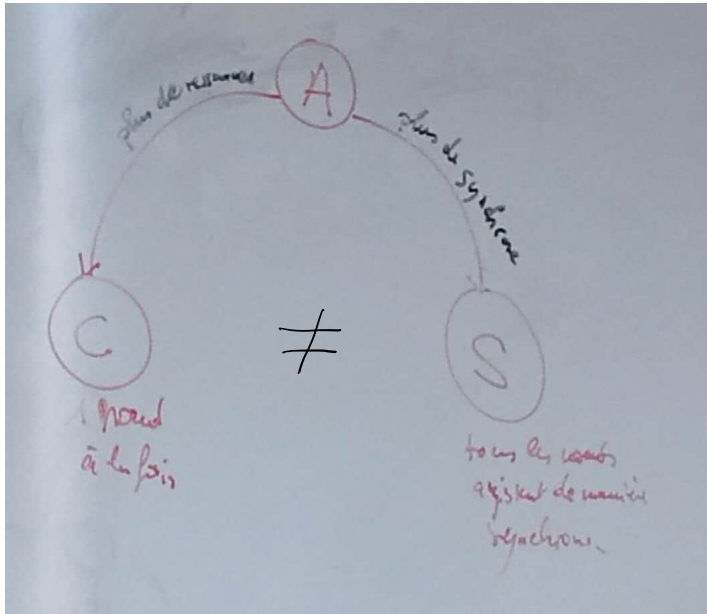
复制 编辑

```
for (i, i, ) {  
    si garde alors Action faire.  
}
```

- 这里 `garde` (守护条件) 是一个节点决定是否执行操作的约束条件。
- 只有当 `garde` 成立时, `Action` (操作) 才会执行。
- 这可能是在讨论**基于守护条件的调度 (guard-based scheduling)**。
- 下面的树形图：
 - `h` 和 `h+1` 可能表示不同时间步或状态。
 - 说明同步 (synchronisation) 在分布式系统中的作用, 所有节点都在协调行动。



$\rightarrow$ synchronise



1. A (Asynchrone) → C (Centralisé)

- "Plus de ressource" (需要更多资源)

- "1 nœud à la fois" (一次只有一个节点执行)

说明: **中心化守护进程 (Centralized Daemon)**, 即一次只允许一个节点进行操作, 这样可以减少同步的开销, 但可能会导致性能瓶颈。

2. A (Asynchrone) → S (Synchrone)

- "Plus de synchronie" (更多的同步性)

- "Tous les nœuds agissent de manière synchrone" (所有节点同步执行)

说明: **同步守护进程 (Synchronous Daemon)**, 即所有节点都遵循一个**全局时钟或同步机制**, 同时执行操作, 保证确定性, 但可能会降低系统的灵活性。

(1) 分布式系统中的不同调度模式

- **中心化调度 (Centralized, C)**

- 只有一个节点 (或进程) 能执行任务。
- 易于控制, 但可能成为瓶颈 (bottleneck)。
- 适用于对一致性要求较高的情况。

- **同步调度 (Synchronous, S)**

- 所有节点按照相同的时间步执行。
- 可以避免竞争条件 (race condition), 但需要高效的同步机制 (如全局时钟)。
- 适用于**时间驱动的分布式计算** (如分布式数据库中的**两阶段提交协议 (2PC)**)。

- **异步调度 (Asynchronous, A)**

- 每个节点独立运行, 不需要全局同步。
- 允许更高的并发度, 但可能引入不确定性和公平性问题 (如资源饥饿)。
- 适用于大规模分布式系统, 如区块链、分布式计算框架 (如Spark、MapReduce)。

• 从 A → C (异步 → 中心化)

- **提高资源利用率, 但降低并行度** (因为一次只允许一个节点执行)。
- 适用于任务间存在依赖关系的情况 (如主从复制模型)。

• 从 A → S (异步 → 同步)

- **提高同步性, 但增加通信开销** (因为需要确保所有节点同时执行)。
- 适用于高一致性要求的场景 (如分布式数据库事务)。

asynchrone + unfair

↓ fairness

???

第4章 本地互斥 (Local Mutual Exclusion)

在本练习中，我们研究本地互斥问题。以下是该问题的规范：

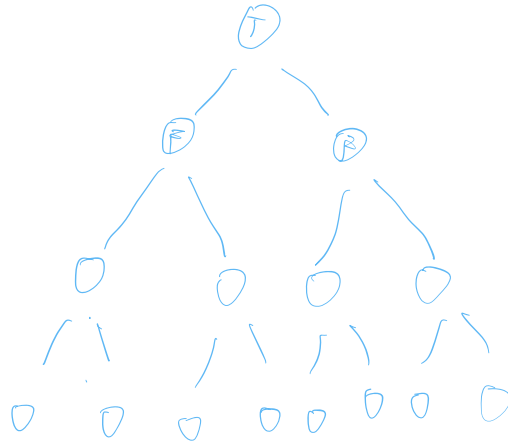
规范 4 (本地互斥)：

- **活性 (Liveness)**：任何处理器都能无限多次进入临界区。
- **安全性 (Safety)**：在任何配置中，相邻的两个节点不能同时处于临界区。

网络为一个有向树，每个处理器 p 可以通过 F_p 访问树中它的父节点，并通过 C_p 访问它的子节点集合。我们假设调度是分布式的弱公平调度 (weakly fair scheduling)。每个处理器 p 都有一个布尔变量 b_p ，算法如下。

问题

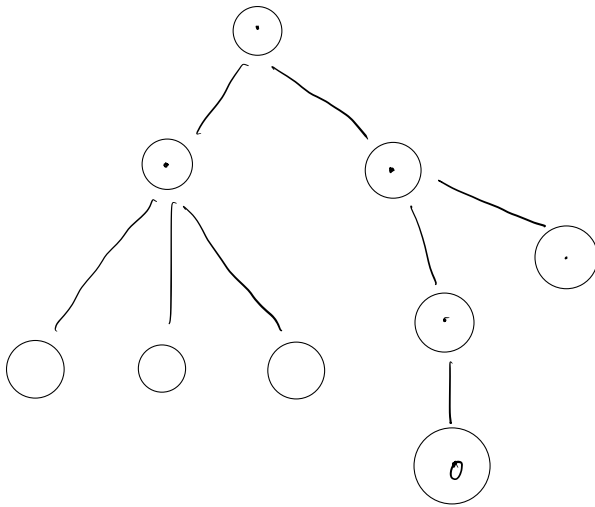
1. 证明在任何配置中，已启用的处理器的邻居都不会被启用。
2. 证明如果在配置 γ 中处理器 p 的值为 b_p ，则 p 的任何子节点都将在有限时间内采取值 b_p 。
3. 证明从任何配置开始，任何处理器都会被无限多次激活。
4. 得出结论。



算法3：用于处理器 p 的自稳定本地互斥

1. **根节点规则 (对于 $p = r$)**：
 $(R) :: \forall q \in C_r, b_r = b_q \rightarrow b_r := \neg b_r$
2. **叶节点规则 (对于 $p = l$)**：
 $(L) :: b_l \neq b_{F_l} \rightarrow b_l := \neg b_l$
3. **内部节点规则 (对于内部节点 p)**：
 $(I) :: (b_p \neq b_{F_p}) \wedge (\forall q \in C_p, b_p = b_q) \rightarrow b_p := \neg b_p$

Q1



supposons qu'il existe une config γ dans laquelle deux nœuds voisins sont simultanément activables à même temps.

son pet générera

① $p = r \Rightarrow C_r = C_q$  y est qu'on a 2 cases

active sur $q \Rightarrow C_r + C_q$.

γ est celle $\in \mathcal{I}$, est
activable

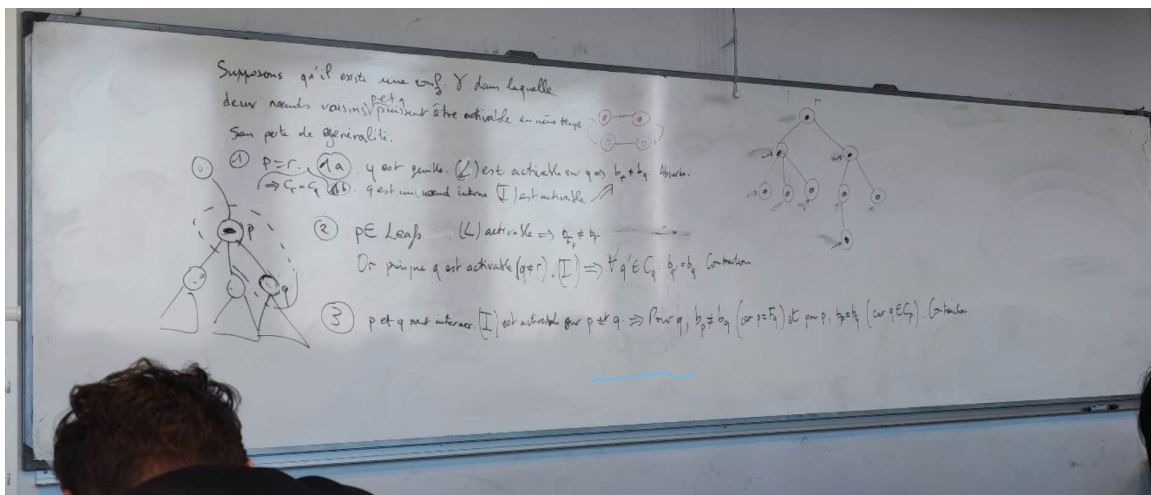
② $p \in \text{leaf}$ $\wedge$ activable $\Rightarrow b_{F_p} \neq b_p$

Or puisque q est activable $\wedge q \neq r$,

$\in \mathcal{I}$, $\Rightarrow \forall q' \in C_q, b_{q'} = b_q$ Contradiction



③ p et q sont internes $\in \mathcal{I}$, voir



Supposons qu'il existe une configuration γ dans laquelle
deux nœuds voisins p et q peuvent être activables en même temps.

Sans perte de généralité :

1. $p = r$.
 $q \in C_r$. (R) est activable sur $p \Rightarrow b_p \neq b_q$. Absurde.
 q est un nœud interne (I) alors activable $\Rightarrow$ contradiction.
2. $p \in \text{Leaf}$.
 (L) activable $\Rightarrow b_p \neq b_{F_p}$. Contradiction.
3. p et q sont internes.
 (I) est activable pour $p \neq q$ (car $p = F_q$) et donc $b_p \neq b_q$ (car $q \in C_p$). Contradiction.

假设存在一个配置 γ , 在该配置中, 两个相邻的节点 p 和 q 可以同时被激活。

不失一般性地分析如下:

1. p 是根节点 r :
 $q \in C_r$, 规则 (R) 对 p 可激活 $\Rightarrow b_p \neq b_q$. 矛盾。 $\rightarrow$ contradiction
 q 是内部节点 (I) , 那么 q 可激活 $\Rightarrow$ 矛盾。
2. p 是叶节点:
 (L) 可激活 $\Rightarrow b_p \neq b_{F_p}$. 矛盾。
3. p 和 q 都是内部节点:
 (I) 对 p 可激活 $\Rightarrow p \neq q$ (因为 $p = F_q$), 因此 $b_p \neq b_q$ (因为 $q \in C_p$). 矛盾。

情况 1: p 是根节点 (Root)

假设 p 是树的根节点 r , 即 $p = r$ 。

- 由于 q 是 p 的子节点 ($q \in C_r$) , 根据**根节点规则 (R)** :

$$b_r = b_q \rightarrow b_r := \neg b_r$$

- 这意味着: 如果 q 处于被激活的状态 (b_q 被翻转) , 那么 b_r 也会被翻转。
- 但是 p 是根节点, 根节点的规则要求 **它的状态与其子节点不同** (即 $b_r \neq b_q$) 。
- 矛盾!**
- 另一种可能性是 q 是一个内部节点 (即遵循内部节点规则 (I)) 。
- 根据内部节点规则, q 被激活的前提是 $b_p = b_q$ (但我们刚才证明了 $b_p \neq b_q$) 。
- 矛盾!**

因此, 在**根节点的情况下, 相邻的两个节点不能同时被激活。**

情况 2: p 是叶子节点 (Leaf)

假设 p 是叶子节点, 即 $p \in Leaf$ 。

- 叶子节点的规则 (L) :

$$b_l \neq b_{F_l} \rightarrow b_l := \neg b_l$$

其中 F_l 是 p 的父节点。

- 如果 p 可被激活**, 则它的布尔变量 b_p **必须与它的父节点 b_{F_p} 不同**。
- 但假设 p 和 q 可以同时被激活, 意味着它们的值都可以翻转。
- 由于 p 和 q 是相邻节点, 它们可能共享相同的父节点 F_p 。
- 这会导致一个矛盾, 因为同一个父节点不能同时允许两个子节点被激活。

因此, 在**叶子节点的情况下, 相邻的两个节点不能同时被激活。**

情况 3: p 和 q 都是内部节点 (Internal Nodes)

假设 p 和 q 都是内部节点。

- 内部节点的规则 (I) :

$$(b_p \neq b_{F_p}) \wedge (\forall q \in C_p, b_p = b_q) \rightarrow b_p := \neg b_p$$
- 这里的关键是:
 - 如果 p 被激活**, 则它的值 b_p 必须与其父节点 F_p **不同**。
 - 同时, 它的值必须和它的所有子节点 q **相同** ($b_p = b_q$) 。
 - 但是我们假设 q 也同时被激活, 意味着 $b_p \neq b_q$ 。
 - 矛盾!**

因此, 在**内部节点的情况下, 相邻的两个节点不能同时被激活。**

算法3: 用于处理器 p 的自稳本地互斥

- 根节点规则 (对于 $p = r$) :**

$$(R) :: \forall q \in C_r, b_r = b_q \rightarrow b_r := \neg b_r$$
- 叶节点规则 (对于 $p = l$) :**

$$(L) :: b_l \neq b_{F_l} \rightarrow b_l := \neg b_l$$
- 内部节点规则 (对于内部节点 p) :**

$$(I) :: (b_p \neq b_{F_p}) \wedge (\forall q \in C_p, b_p = b_q) \rightarrow b_p := \neg b_p$$

结论

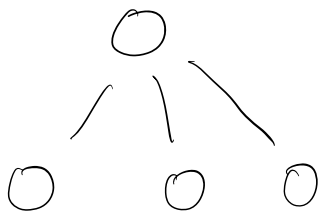
通过以上三种情况的讨论, 我们发现如果假设**两个相邻节点可以同时被激活**, 都会导致**矛盾**。

因此, 我们证明了:

在任何配置中, 已启用的处理器的邻居不会被启用。

这就是**安全性 (Safety) **的保证。

Q2



Rem :

p ne peut pas être une feuille.

1. p est activable $\Rightarrow$ trivialement prouvé que p soit r ou interne.

En effet, en γ , on a bien $b_p = b_q, q \in C_p$.

2. p n'est pas activable et il existe $q \in C_p$, tel que $b_q \neq b_p$.

- q est une feuille $\Rightarrow$ puisque $b_p \neq b_q$, (L) est activable sur q .

Remarque : p est bloqué tant que q n'a pas exécuté (L) .

Par équité, le démon finit par faire exécuter (L) sur q . $\Rightarrow$ dans toute exécution à partir de γ , il existe une configuration où $b_q = b_p$.



备注:

节点 p 不能是叶节点。

1. p 可激活:

这显然证明 p 是根节点 r 或内部节点。

因为在配置 γ 中, $b_p = b_q$, 其中 $q \in C_p$ 。

2. p 不可激活:

存在 $q \in C_p$, 使得 $b_q \neq b_p$ 。

- 如果 q 是叶节点: 因为 $b_p \neq b_q$, 规则 (L) 对 q 可激活。

备注: p 在 q 未执行 (L) 前会被阻塞。

由于公平性, 调度器最终会对 q 执行 (L) 。

因此, 在从配置 γ 开始的所有执行中, 存在一个配置满足 $b_q = b_p$ 。

Cas 1 : q n'est pas une feuille.

Cas 2 : $b_q = 1$ (les feuilles de l'arbre de q). $\Rightarrow \forall q' \in C_q, q'$ est une feuille.

Donc $\forall q' \in C_q$ tel que $b_{q'} = b_q$, (L) est activable.

Par fairness, $\forall q' \in C_q$ tel que $b_{q'} \neq b_q$, $b_{q'}$ finit par être égale à b_q .

情况1: q 不是叶节点。

情况2: $b_q = 1$ (这是 q 子树的叶节点的值)。

$\Rightarrow \forall q' \in C_q, q'$ 是叶节点。

因此, 对于所有满足 $b_{q'} = b_q$ 的 $q' \in C_q$, 规则 (L) 是可激活的。

由于公平性, 对于所有满足 $b_{q'} \neq b_q$ 的 $q' \in C_q$, 最终 $b_{q'}$ 将等于 b_q 。

第一步: 先考虑 p 的激活情况

- 如果 p 可被激活 (即规则允许它翻转), 那么 p 可能会改变值。
- 如果 p 不可被激活, 则它当前的值是稳定的, 不会再次改变。
- 由于我们要证明 q 会最终等于 p , 我们假设 p 已经稳定, 即 b_p 不再变化。

第三步: 保证有限时间内收敛

- 由于调度器保证了公平性 (Fairness), 意味着每个可以被激活的节点, 最终都会被执行规则。
- 这确保了不会有节点永远不被更新, 因此最终所有子节点都会更新为 $b_q = b_p$ 。

第二步: 如果 p 不可被激活, 它的子节点 q 可能会变化

- 假设在配置 γ 中, 存在一个子节点 q 使得 $b_q \neq b_p$ 。
- 根据规则, 我们要看 q 是否能够被激活。

情况 1: q 是叶子节点

- 叶子节点遵循规则:

$$(L) :: b_l \neq b_{F_l} \rightarrow b_l := \neg b_l$$

- 由于 $b_q \neq b_p$ (即 q 和它的父节点 p 值不同), 所以 q 可以被激活。
- 因此, 调度器最终会执行 L 规则, 使得 $b_q := b_p$, 即 q 变得和 p 一致。

情况 2: q 不是叶子节点

- 设 q 也是一个内部节点, 它可能有自己的子节点 C_q 。
- 递归地应用相同的逻辑:
 - 如果 q 的所有子节点 q' 都已经和 q 相同, 则 q 可能会翻转。
 - 但如果至少有一个子节点不同于 q , 则 q 仍然不能翻转, 必须等待它的子节点先翻转。
- 通过类似的推导, 叶子节点首先会执行 L 规则, 并逐步影响上层节点, 直到整个树达到稳定状态。

3. 总结

- 如果 p 不可被激活 (即它的值稳定), 则所有不同于 p 的子节点 q 终究会翻转, 以匹配 p 的值。
- 这个过程可能需要多步迭代, 但由于系统的公平性, 所有子节点最终都会变得和父节点 p 一致, 即 $b_q = b_p$ 。
- 因此, 我们证明了:

在有限时间内, 所有子节点 q 的值都将变为 b_p 。

问题

- 证明在任何配置中, 已启用的处理器的邻居都不会被启用。
- 证明如果在配置 γ 中处理器 p 的值为 b_p , 则 p 的任何子节点都将在有限时间内采取值 b_p 。
- 证明从任何配置开始, 任何处理器都会被无限多次激活。
- 得出结论。

算法3: 用于处理器 p 的自稳定本地互斥

- 根节点规则 (对于 $p = r$) :
(R) :: $\forall q \in C_r, b_r = b_q \rightarrow b_r := \neg b_r$
- 叶节点规则 (对于 $p = l$) :
(L) :: $b_l \neq b_{F_l} \rightarrow b_l := \neg b_l$
- 内部节点规则 (对于内部节点 p) :
(I) :: $(b_p \neq b_{F_p}) \wedge (\forall q \in C_p, b_p = b_q) \rightarrow b_p := \neg b_p$

Q3.

On suppose que l'assertion est vraie pour tout q tel que $h_q = H$.

On prouve que $h_q = H + 1$.

Pour chaque enfant q' de q , par l'hyp de récursivité, tous ses enfants finissent par avoir la couleur de q .

Donc chaque enfant q' de q finit par pouvoir être activable.

De la même manière que précédemment, tous les enfants de q finissent par avoir la même couleur que q .

Par contradiction, on suppose qu'il existe s'exécution en 1 mod p au cours de laquelle p finit par être bloqué.

1. $p = r$.
Pour que p soit bloqué, il faut qu'il ne puisse plus jamais exécuter (R).
Or, par la question, on sait tous ses enfants finissent par avoir la même couleur que lui.
Donc, p finit par pouvoir exécuter (R). Contradiction.
2. p est interne.
Soit p l'un des nœuds bloqué les plus proche de r .
 $\Rightarrow$ tous les nœuds entre r et p ne sont pas bloqués.
Or q , tous les enfants de p finissent par être de la même couleur que p .
Puisque p est en haut, son parent n'est pas bloqué et par conséquent, p change infiniment souvent de côté.

1. 假设 p 最终被阻塞

- 假设在某次执行过程中, 存在某个处理器 p 最终不会再被激活 (即它被永久阻塞)。
- 我们来分析不同情况下, p 是否真的会被阻塞。

2. 情况 1: p 是根节点 ($p = r$)

- 假设 p 被阻塞, 那么它就无法再执行根节点规则 R :

$$(R) :: \forall q \in C_r, b_r = b_q \rightarrow b_r := \neg b_r$$

- 但根据第二题, 我们已经证明所有的子节点最终都会变得和 p 相同, 即 $b_q = b_p$ 。
- 这样一来, 根节点的规则 R 必然会被触发, 使得 p 被激活并翻转状态。
- 这与我们的假设 (p 永远不被激活) 矛盾!
- 因此, 根节点不会被永久阻塞。

假设对于所有满足 $h_q = H$ 的 q , 断言成立。

我们证明对于 $h_q = H + 1$ 的情况, 该断言也成立。

对于 q 的每个子节点 q' , 根据递归假设, 它的所有子节点最终会拥有与 q 相同的颜色。
因此, q 的每个子节点 q' 最终能够被激活。

类似于之前的情况, q 的所有子节点最终都会拥有与 q 相同的颜色。

通过反证法, 假设存在一个执行, 其中处理器 p 最终被阻塞。

1. $p = r$:
如果 p 被阻塞, 那么 p 就无法再执行规则 (R)。
然而, 根据问题描述, 我们知道 p 的所有子节点最终会与其具有相同的颜色。
因此, p 最终能够执行规则 (R)。这与假设矛盾。
2. p 是内部节点:
假设 p 是离根节点 r 最近的被阻塞节点之一。
 $\Rightarrow$ 位于 r 和 p 之间的所有节点都没有被阻塞。
另一方面, p 的所有子节点最终都会与 p 具有相同的颜色。
因为 p 位于树的上层, 其父节点未被阻塞, 因此 p 会无限次改变其状态。

3. 情况 2: p 是内部节点

- 假设 p 是离根节点最近的被阻塞的内部节点, 即在路径上, r 到 p 之间的所有节点都是可以被激活的。
- 另一方面, p 的所有子节点最终会和它的值相同 (根据第二题的证明)。
- 那么 p 为什么不能被激活?
 - 因为内部节点规则是:
$$(I) :: (b_p \neq b_{F_p}) \wedge (\forall q \in C_p, b_p = b_q) \rightarrow b_p := \neg b_p$$
 - 由于 p 的所有子节点最终都会和它一致, 意味着如果它的父节点 F_p 允许, 它就可以被激活。
 - 但是 p 是离根节点最近的阻塞节点, 所以它的父节点是可以被激活的, 不会一直阻塞。
 - 这意味着** p 也会在某个时刻被激活**, 这与它“永久阻塞”的假设矛盾!

4. 结论

由于根节点不会被永久阻塞, 并且任何内部节点如果它的父节点是可被激活的, 它最终也会被激活, 所以我们得出结论:

任何处理器都会被无限多次激活, 不会永久阻塞。

这就证明了系统具有活性 (Liveness), 即:

- 不会有节点永久被卡住。
- 所有节点都会在有限时间内被激活并执行操作。

4.1 练习 5: 着色 (Coloring)

在本练习中，我们研究 着色 (Coloring) 问题。该问题的规范如下：

规范 5 (着色)

- 活性 (Liveness)：该协议将在有限时间内终止。
- 安全性 (Safety)：在终态配置 (terminal configuration) 中，每个处理器都持有一个颜色，并且任何两个相邻的处理器不会持有相同的颜色。

网络模型

网络是一个任意图 (arbitrary graph)。我们假设采用中心化的弱公平调度 (central weakly fair scheduling)。

每个处理器 p 具有一个变量 c_p ，其取值范围为集合 $\{0, \dots, \Delta\}$ 。

我们定义距离至少为 2 的、远离拜占庭处理器 (Byzantine processors) 的处理器集合为 V^* 。

对于任何正确的处理器 p ，我们定义以下谓词：

- colored_p ：

$$\forall q \in N_p, c_q \neq c_p$$

即：p 的所有邻居都具有不同的颜色 (p 是正确着色的)。

- faulty_p ：

$$\exists q \in N_p, c_q = c_p$$

即：p 具有与至少一个邻居相同的颜色 (p 处于错误着色状态)。

对于任意配置 γ 和处理器 p ，可以容易验证，这两个谓词中，恰好有一个对 p 成立。

算法

算法 4: 严格自稳定着色 (Strict-stabilizing coloring)

适用于处理器 p 的规则：

$$(C) :: \exists q \in N_p, c_p = c_q \longrightarrow c_p := \text{choose}(\{0, \dots, \Delta\} \setminus \{c_q | q \in N_p\})$$

解释：

- 如果处理器 p 的颜色与邻居 q 相同 (即 p 违反了着色约束)，
- 则 p 选择一个新的颜色，该颜色必须是 $\{0, \dots, \Delta\}$ 集合中不同于所有邻居的颜色。

问题

1. 证明：对于集合 V^* 中的任何处理器 p ，如果它在某个配置 γ 中满足 colored_p ，那么在从 γ 开始的任何执行过程中，它始终满足 colored_p 。
2. 证明：对于集合 V^* 中的任何处理器 p ，如果它在某个配置 γ 中满足 faulty_p ，那么它将在有限时间内满足 colored_p 。
3. 总结结论 (Conclude)。